

Presentation 4

STAT 390 | Winter 2026 | Team 6

Northwestern

Programming

Vicky Xu

Task 2: Effect of patch order on accuracy

Core Question: Since the batch size is 1, does the random order of patches during training significantly change the model's final diagnosis?

Approach: Ran the code for 5 different data splits, using 10 different random seeds per split (Total 50 experiments).

Task 2: Effect of patch order on accuracy

1. Capture Seed and Best Validation Loss
 - problem: The standard output did not explicitly record which seed produced which result in the final results.txt.
 - solution: Updated main.py to record args.seed and best validation loss in results.txt for each run.

[main.py](#)

```
# Save final results
results_path = os.path.join(run_dir, "results.txt")
with open(results_path, 'w') as f:
    #task2 modification
    f.write(f"Seed: {args.seed}\n")
    f.write(f"Best Val Loss: {trainer.best_val_loss:.4f}\n")
```


Task 2: Effect of patch order on accuracy

[main.py](#)

[util.py](#)

2. Unique Directory Naming (Collision Avoidance)

- problem: Originally, the code created folders based only on a timestamp (e.g., 20260312_1500). Because Slurm Array Jobs launch multiple tasks in the same second, many tasks tried to write to the same folder, causing data to be overwritten.
- solution: Modified `utils.py` and `main.py` to include the Slurm Task ID and Random Seed in the folder name (e.g., 20260312_1500_TaskID5_Seed40).

```
def create_run_directory(name: str = None, base_dir: str = None) -> str:
    """Create a unique run directory with timestamp and a unique name"""
    from datetime import datetime
    from config import DATA_PATHS

    if base_dir is None:
        base_dir = DATA_PATHS['runs_dir']

    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")

    # Use the 'name' passed from main.py, or fall back to SLURM_JOB_NAME
    if name is None:
        name = os.environ.get('SLURM_JOB_NAME', 'run')
    # This creates (eg): runs/20260312_1600_MIL_Task2_ID0_Seed10
    run_dir = os.path.join(base_dir, f"{timestamp}_{name}")

    os.makedirs(run_dir, exist_ok=True)

    print(f"Created run directory: {run_dir}")
    return run_dir
```

```
> 20260312_160914_MIL_Task2_ID0_Seed10
> 20260312_160914_MIL_Task2_ID1_Seed20
> 20260312_160914_MIL_Task2_ID2_Seed30
> 20260312_160914_MIL_Task2_ID3_Seed40
```

[/runs](#)

Task 2: Effect of patch order on accuracy

[run_task2.sbatch](#)

3. Modify the sbatch file

- Launch 50 parallel experiments with a single command, ensuring that all five data splits were tested across ten different random seeds to observe the effects of non-convex optimization and patch ordering.

```
# 1. compute Split and Seed
SPLIT_NUM=$(( (SLURM_ARRAY_TASK_ID / 10) + 1 ))
SEED_IDX=$(( SLURM_ARRAY_TASK_ID % 10 ))

# 2. define 10 seed
SEEDS=(10 20 30 40 50 60 70 80 90 100)
CURRENT_SEED=${SEEDS[$SEED_IDX]}
SPLIT_FILE="./no_train_val_leakage_splits/data_splits_new_0${SPLIT_NUM}.npz"

echo "-----"
echo "Job ID: $SLURM_JOB_ID | Task ID: $SLURM_ARRAY_TASK_ID"
echo "Running $SPLIT_FILE with Seed $CURRENT_SEED"
echo "Start time: $(date)"
echo "-----"

# 3. run task2 command
python3 main.py \
    --load_splits "$SPLIT_FILE" \
    --seed "$CURRENT_SEED" \
    --epochs 30
```

Task 2: Effect of patch order on accuracy

4. Data Visualization

[task2.ipynb](#)

- Collect data from all 50 unique directories. Parse performance metrics from text files. Merge CSV prediction data into a unified format for plotting.
- Best Validation Loss Boxplot
- Predicted Probabilities Boxplot
- Classification Consistency Bar Plot
- Accuracy Variability Table

```
In [3]: df_summary.head()
```

```
Out[3]:
```

	actual_id	split	seed	best_val_loss	test_acc	folder
0	0	Split 1	10	0.1418	0.9444	20260312_160914_MIL_Task2_ID0_Seed10
1	1	Split 1	20	0.0319	0.8889	20260312_160914_MIL_Task2_ID1_Seed20
2	2	Split 1	30	0.1584	0.9444	20260312_160914_MIL_Task2_ID2_Seed30
3	3	Split 1	40	0.1872	0.8889	20260312_160914_MIL_Task2_ID3_Seed40
4	4	Split 1	50	0.0603	0.9444	20260312_160914_MIL_Task2_ID4_Seed50

```
In [4]: df_all_preds.head()
```

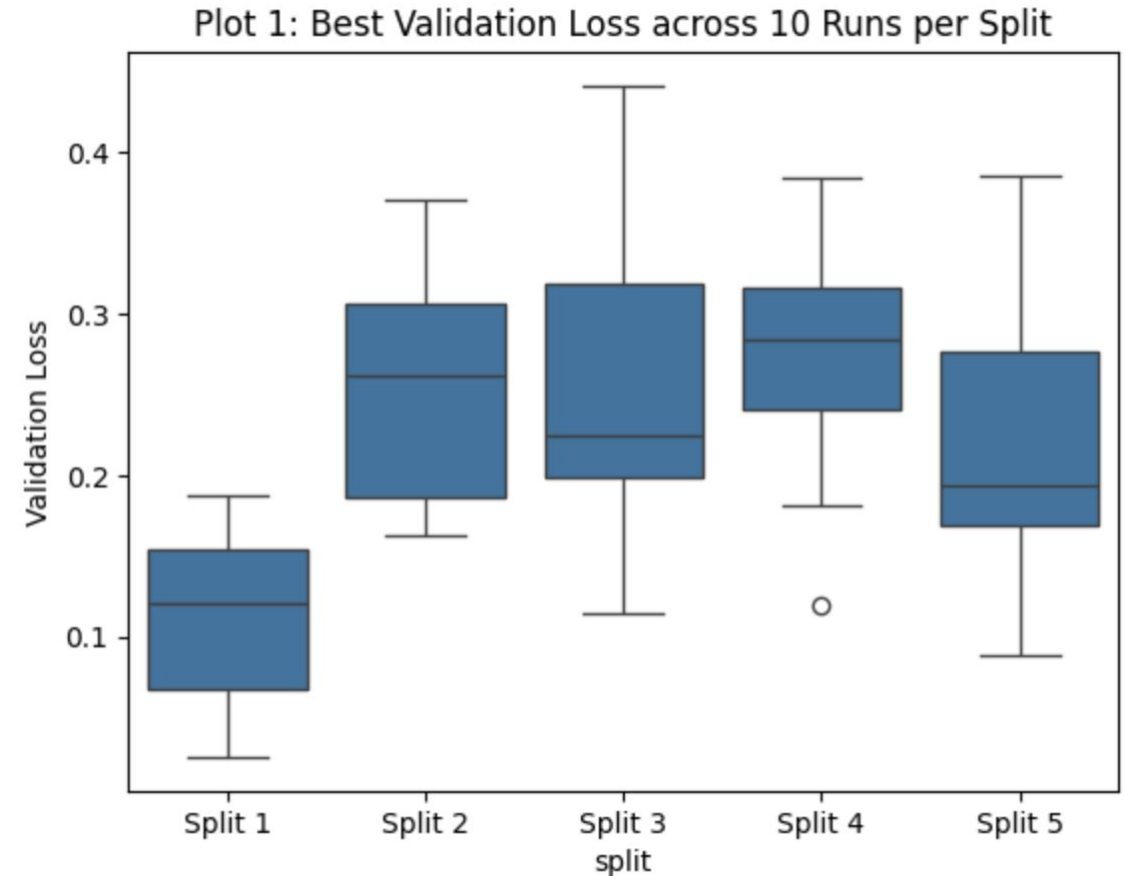
```
Out[4]:
```

	case_id	true_label	predicted_label	correct	prob_class0	prob_class1	split	actual_id
0	7	1	1	True	5.511316e-09	1.000000	3	27
1	15	1	1	True	6.713946e-03	0.993286	3	27
2	22	0	0	True	9.998674e-01	0.000133	3	27
3	28	0	0	True	9.995955e-01	0.000405	3	27
4	38	1	1	True	2.777217e-05	0.999972	3	27

Result: Best Validation Loss across 10 Runs per Split

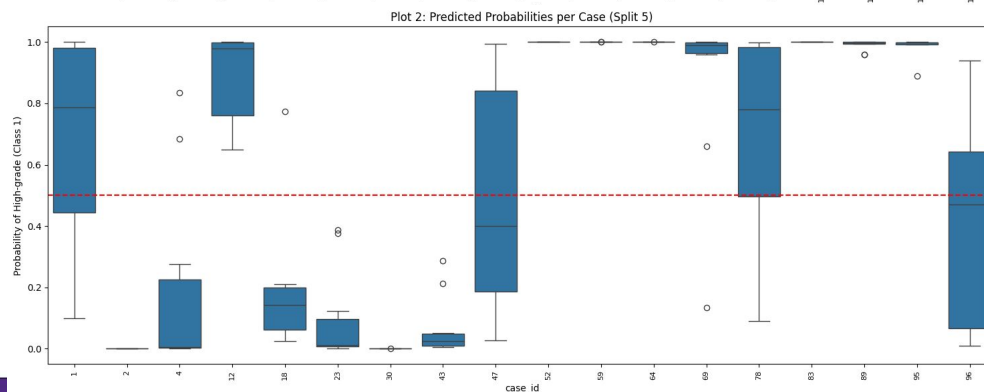
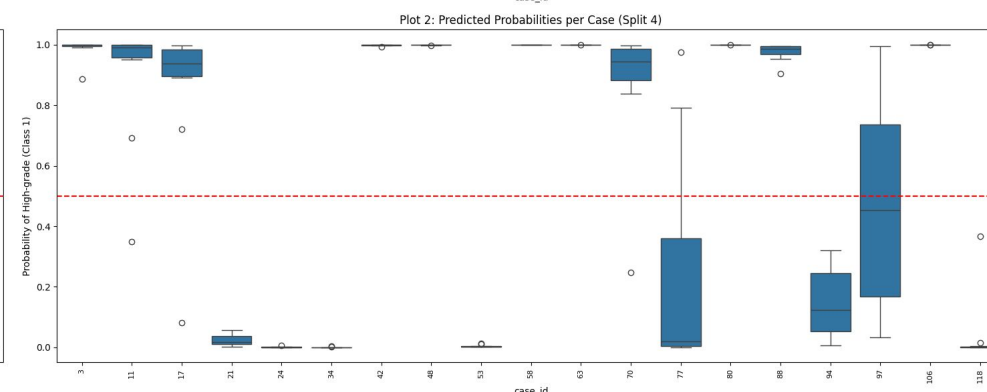
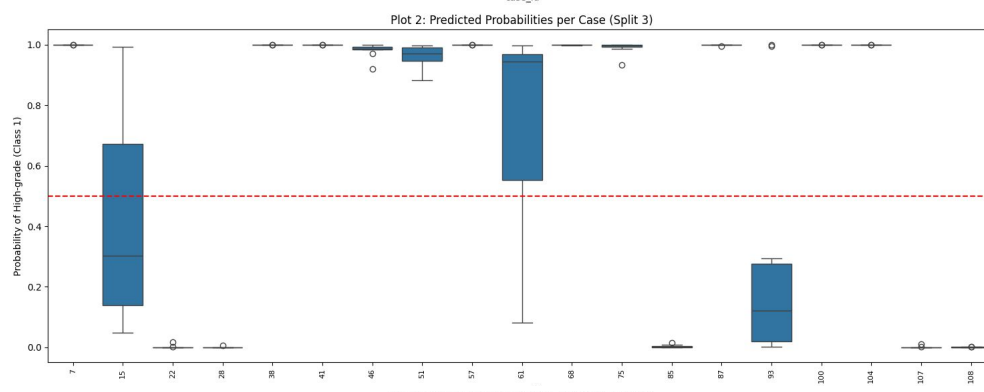
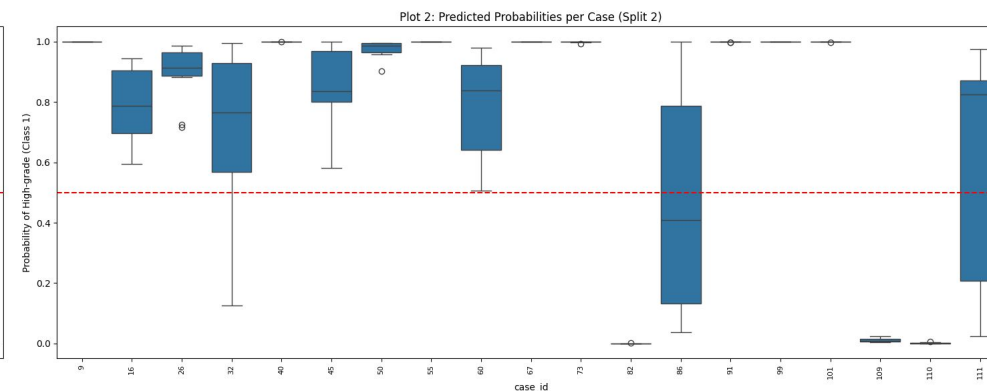
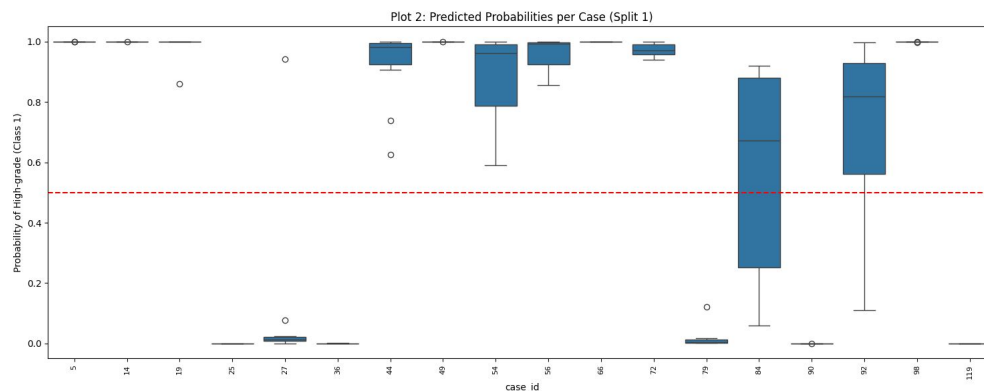
- **Observation of Variability:** Significant vertical spreads in boxplots (especially Split 3) indicate that the final loss is highly sensitive to random initialization and patch ordering.
 - **Outlier Impact:** Presence of outliers in Split 4 highlights instances where specific random seeds led the model into poor local optima.
- > The high variance across runs confirms the non-convex nature of the MIL loss function; with a batch size of 1, the gradient descent trajectory is easily influenced by the stochastic shuffling of patches.

[task2.ipynb](#)



Result: Predicted Probabilities per Case

[task2.ipynb](#)

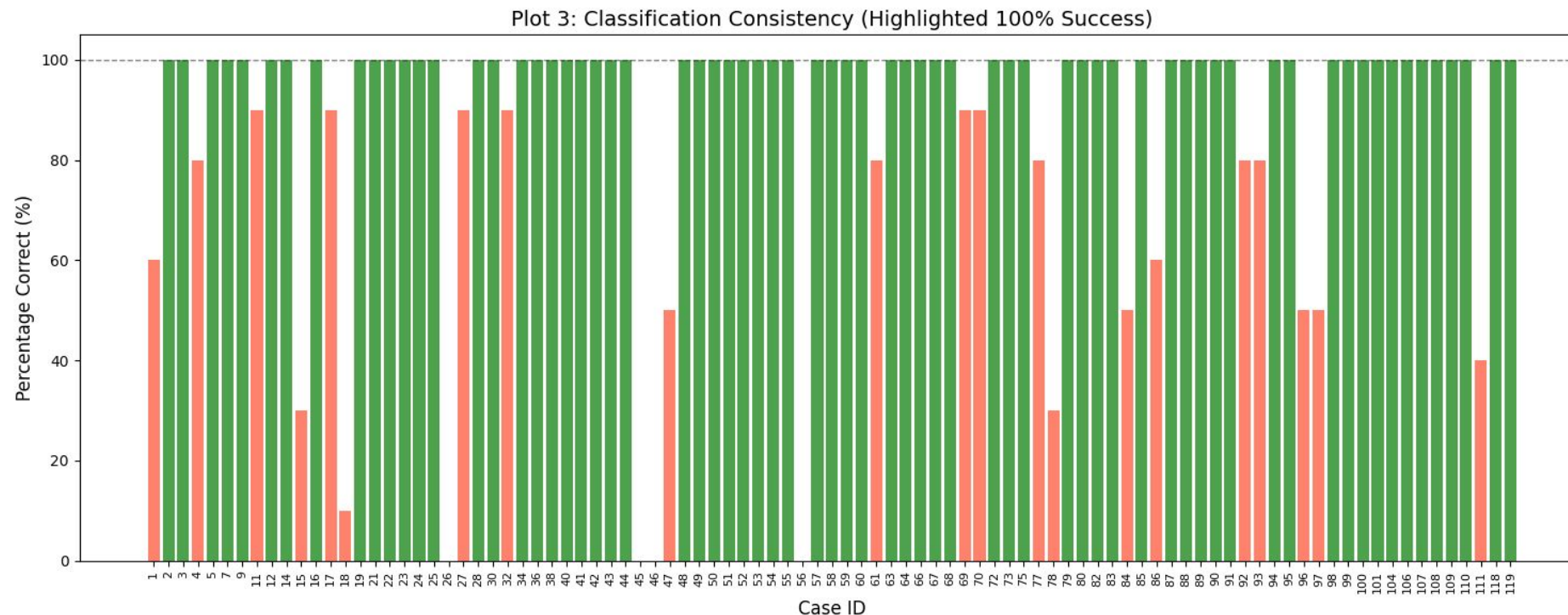


- **Stable Cases:** Narrow boxes near 0.0 or 1.0 indicate high model certainty and robustness to patch ordering.
- **Borderline Cases:** Boxes crossing the 0.5 threshold (red dashed line) identify "unstable" patients whose diagnosis flips based on the random seed.

-> Large variance in probability for certain cases shows that the model's final decision is dependent on the gradient descent trajectory determined by the specific shuffle of training patches. The wide distribution for several cases demonstrates that the MIL loss function is highly non-convex.

Result: Predicted Probabilities per Case

- **Green Bars (100% Success):** Cases that were correctly classified in every single run regardless of the random seed. This represents the model's "robust" knowledge base.
 - **Red Bars (<100% Success):** Cases where the model's accuracy fluctuate based on the random seed.
- > The presence of multiple red bars confirms that diagnostic outcomes for borderline cases are sensitive to the random shuffling of patches during training.



[task2.ipynb](#)

Result: Predicted Probabilities per Case

Impact of Stochasticity:

- Gaps between min and max (e.g., Split 5: 77.8% to 88.9%) show that the final model performance is sensitive to the initial random state and patch presentation order.
- Split 4 achieved a perfect 1.0000 accuracy with a specific seed, yet dropped to 0.8947 with another, highlighting the "luck of the draw" in stochastic training.

[task2.ipynb](#)

Numerical Summary of Accuracy Variability:

	split	min	max
0	Split 1	0.8333	0.9444
1	Split 2	0.7895	0.8947
2	Split 3	0.8421	0.9474
3	Split 4	0.8947	1.0000
4	Split 5	0.7778	0.8889

Result & Next Step

The results demonstrate that patch ordering is a critical hyperparameter in MIL models with small batch sizes. The high variability in accuracy (up to 15% difference) and the existence of inconsistent case classifications prove that the model is sensitive to the stochastic nature of the training process. This highlights the necessity of ensemble learning or multi-seed averaging to ensure stable and reproducible medical diagnoses.

Coder Task 2 + Misc.

Henry Chau

Reflection

- Key learnings
 - CNN training, the effects of initialization, attention, and regularization on predictions
 - Transformations and dropout prevented model from memorizing data
 - More knowledgeable of Quest and Slurm's capabilities
 - Git logistics within a larger shared repository and same local repository
- What I would do differently for a similar project
 - Try to develop the more impactful model changes, mostly worked on non-model part
 - Set class git expectations ahead of time to avoid mishaps, maybe use sparse clones
- Applications to real-world data science projects
 - Trust but verify that previous code/work is correct
 - AI tools were very helpful to quickly evaluate and write code from high-level ideas

Coder Task 2

- I completed task 2 independently from Vicky and initially I created a [shell script](#) that would pass on seed and split specifications to the [sbatch file](#)
- While starting 50 sbatch scripts is possible and allows for more fine tune control of configs, specifically job-name, it is not optimal for the Slurm scheduler so in version 2 I used one [sbatch file](#) with job arrays
- There were much of the same issues as in the previous section but with different solutions
- Instead of writing all training data to a text file, results were put in a json file for easier processing [↗](#)
- Instead of using timestamp with job name I replaced timestamp with job id and used UUID as backup [↗](#)

```
#!/bin/bash

# Exit immediately if a command exits with a non-zero status
set -e

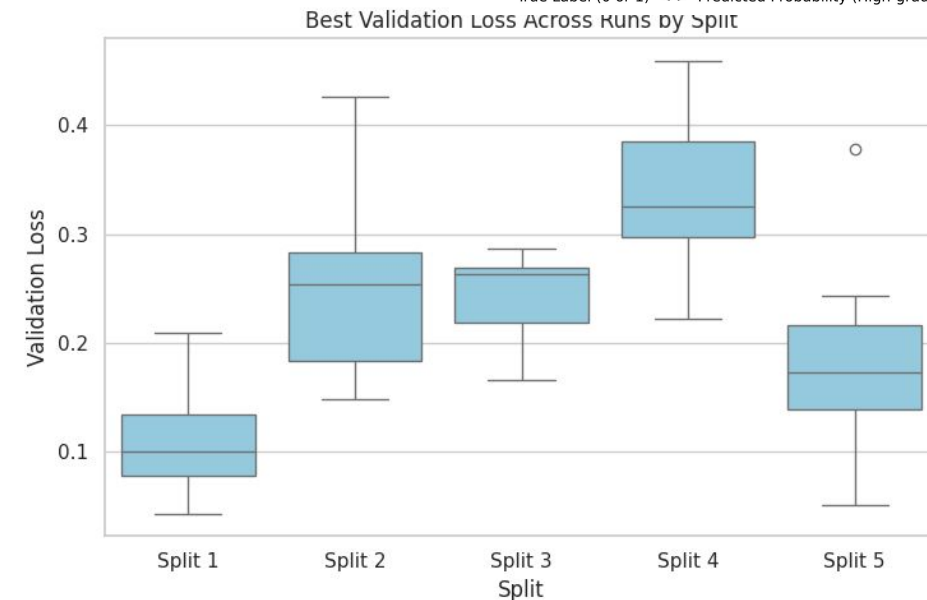
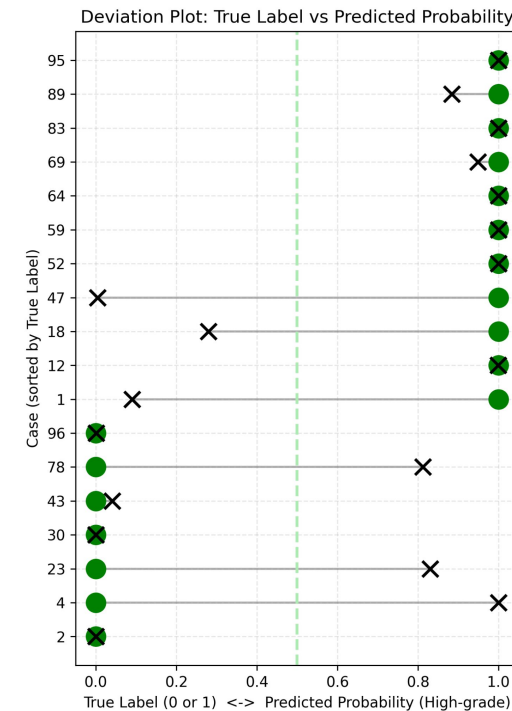
# Seeds to pass for each run
seeds=(1 4 9 16 25 36 49 64 81 100)
# Define splits to iterate through
splits=(1 2 3 4 5)

for seed in "${seeds[@]"; do
    for split in "${splits[@]"; do
        echo "Submitting: Split $split, Seed $seed"

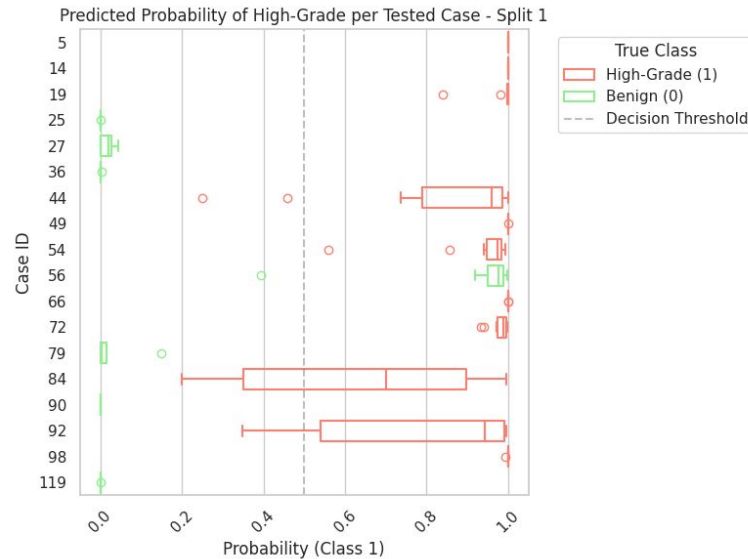
        sbatch --job-name="split${split}_seed${seed}" \
            "sbatch_files/run_splits_10x.sbatch" "$split" "$seed"
    done
done
```

Coder Task 2 Results

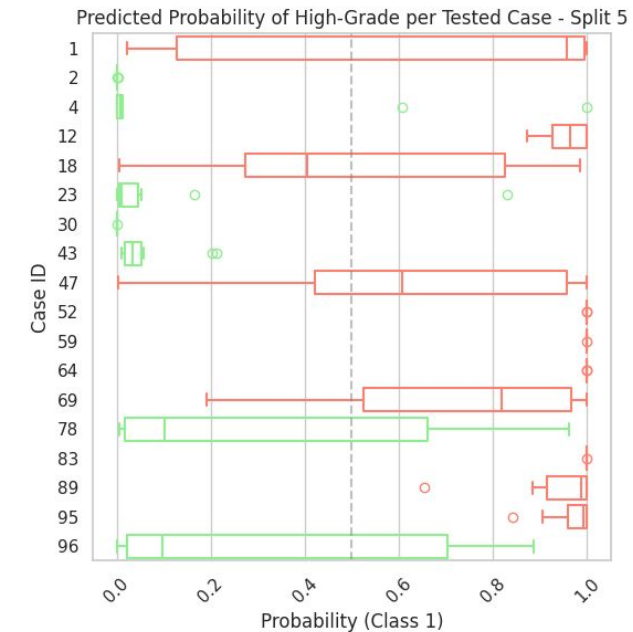
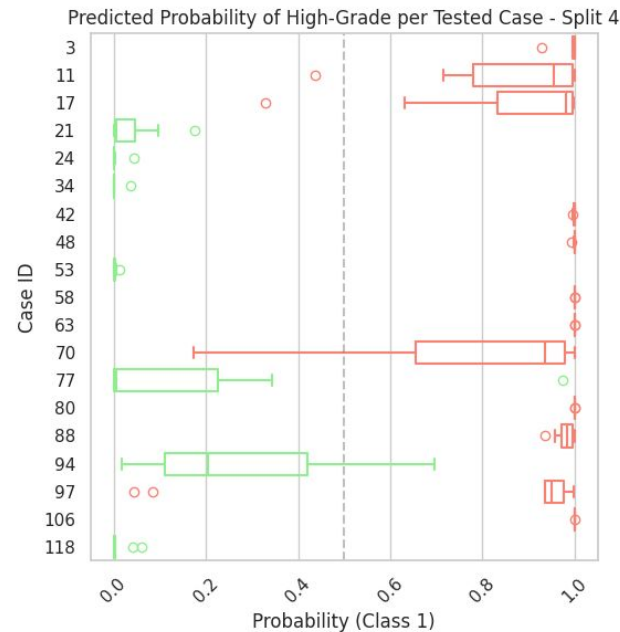
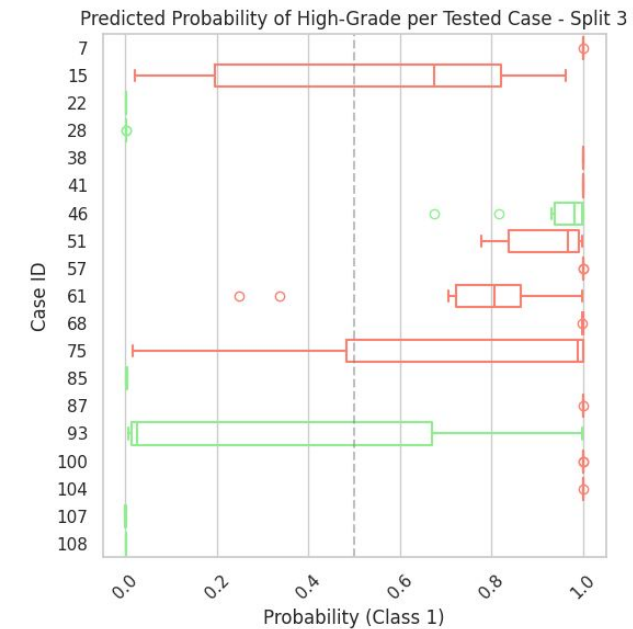
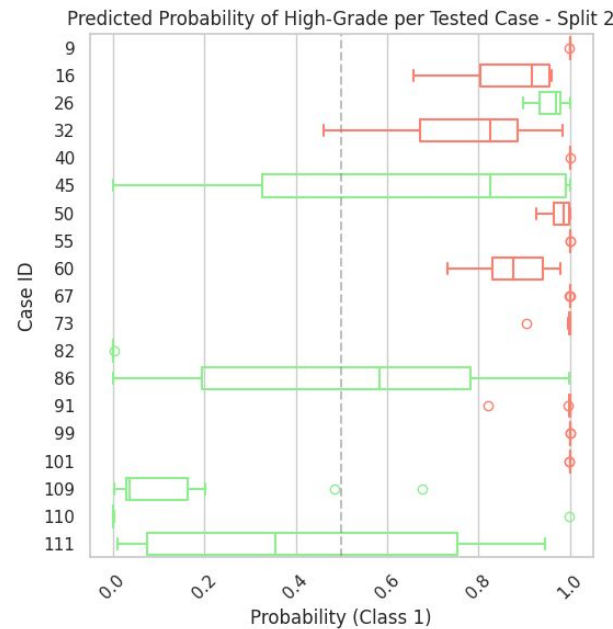
- The boxplots and [log file](#) were created by parsing the results.json from each training run's directory with [summarize_runs.py](#) called from [summarize_runs.sbatch](#) with the [runs folder](#) as target directory
- Split 1 had lower best val loss and 90% average test accuracy
- Split 4 had higher best val loss and yet 95% average test accuracy
- Lowest test accuracy 67% was with split 5, seed 100 [↗](#)
- Other seeds from split 5 show similar issues to a lesser extent suggesting these cases may be hard to classify
- Split 4 seed 4, 49, 64, 100 all achieved 100% accuracy
- Overall the test accuracy seems to be close to 90% with outliers in the lower direction



Coder Task 2 Results [↗](#)

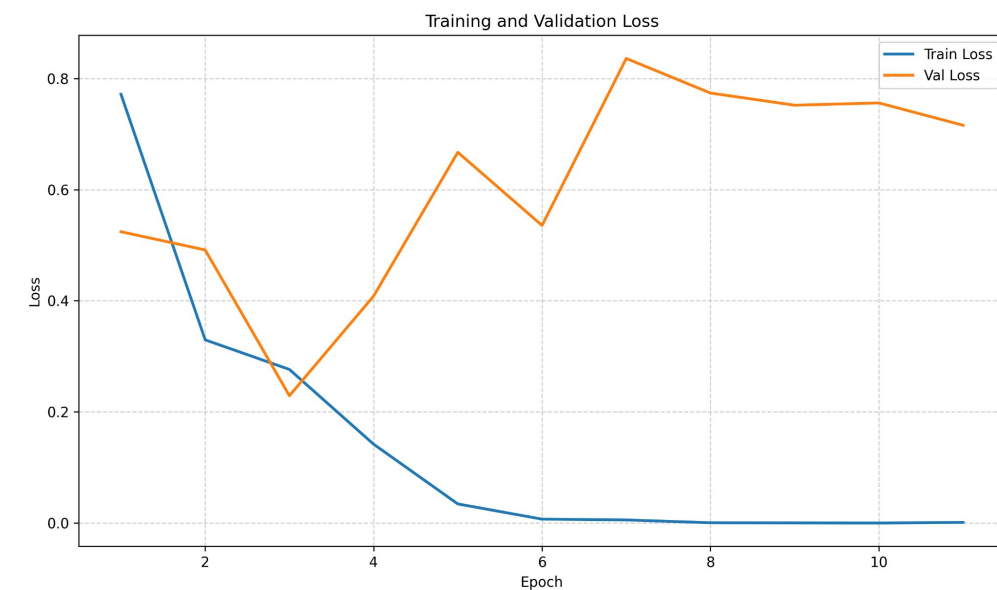


- Predicted probabilities are unstable for case 1, 15, 18, 45, 47, 69, 75, 78, 84, 86, 92, 93, 96, 111 (30–70% correct)
- Case 26, 46, 56 are **consistently being predicted incorrectly**, all benign cases
- Case 26 is a pseudocase with 109, 110, 111 but 109 and 110 do not have issues



Coder Task 2 (extended)

- To differentiate to my task 2 I also looked into whether the model would be better at an earlier epoch since I already have the core structure ready to reuse
- We reported the best validation loss without evaluating the model at that point, the plot of split 2 seed 1 motivated the question whether the model could be more accurate at epoch 3 instead of 11
- This [shell script](#) finds the best checkpoint per run and reevaluates at that point and summarize_runs.py was applied on the [runs directory](#) of reevaluated models



```
#!/bin/bash

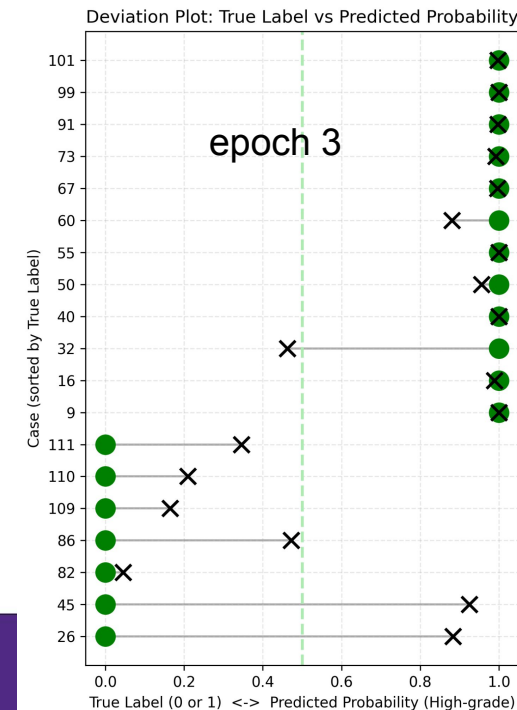
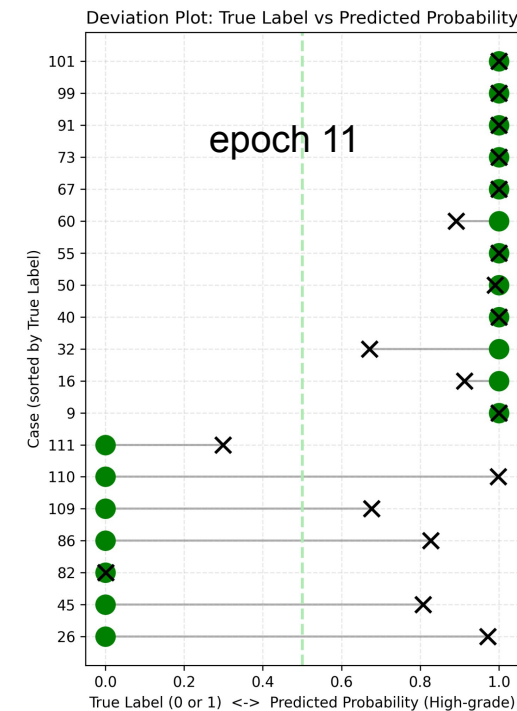
# Exit immediately if a command exits with a non-zero status
set -e

# get filepaths for best checkpoint
readarray -t bests < <(find runs_seed_analysis/ -path '*best.pth')

for best in "${bests[@]"; do
    split=$(echo "$best" | grep -oP "split\K[0-9]+")
    seed=$(echo "$best" | grep -oP "seed\K[0-9]+")
    echo "Submitting: Split $split, Seed $seed, Checkpoint $best"
    sbatch --job-name="split${split}_seed${seed}" \
        "sbatch_files/run_eval_best.sbatch" \
        "$split" "$seed" "$best"
done
```


Coder Task 2 Results (extended) ↗

- For split 2 seed 1, yes accuracy improved 74% $\Rightarrow$ 84%
- However, this improvement is specific to this split and seed, the majority of models did not have an increasing val loss and so average accuracy is largely unchanged
- Original average accuracy: .8858, Best val loss average accuracy: .8829
- Predicted probability plots are similar to before but shows less confidence
- Possible explanations for this result:
 - Model has a flat validation loss curve $\Rightarrow$ performance at the same level
 - Sharp dip in val loss $\Rightarrow$ not applicable to test set, too early and underfit
 - Best val loss when train loss $\approx 0 \Rightarrow$ model unchanged by small gradient
- Could be worth exploring deciding at what point is the model “the best” based on moving average validation loss



Coder Task 2 Summary

- Because the batch size is 1, the specific ordering of the patches leads to local optimums so multiple initializations may be needed to find the best model
- An additional consideration is the data splits themselves, we saw the model performance vary **by seed and by split** as combinations of cases in the training set may influence what features the model learns and thus its performance on the testing set
 - In some splits, the training set and testing set may be more similar or easier to classify
- A more comprehensive study could be done to test each case with different training cases
- To facilitate future exploration, several changes have been made to the code to have better logging, execute with Slurm job arrays in mind, and make randomized data splits as opposed to the deterministic approach currently used

Quick Code Updates

- Better loggings
 - Timestamps are no longer unique, so replaced with Slurm job id or random hex [↗](#)
 - Reporting more statistics such as best val loss, high-grade recall, low-grade recall to results.txt and even more including runtime arguments for reproducibility into machine parseable results.json [↗](#)
 - Automatically generates deviation plot and model loss plot from persistent instance history, now works even if resuming from checkpoint [↗](#)
 - Fixed minor description error mentioned in presentation 1 [↗](#)
 - Fixed missing stain not reporting error [↗](#), logs now report that case 21 sox10 is missing because it is not present the patches directory '/projects/e32998/patches_benign_split' that is used to map to the embeddings (where it does exists). Additionally case 16 melan&sox10 and case 17 melan are completely missing from both [↗](#)

Quick Code Updates

- Data splitting improved
 - Shuffling of cases functionality reinstated to StratifiedGroupKFolds to create more randomized non-overlapping splits (deterministic behavior still default) [↗](#)
 - Initially was meant to be included in presentation 3 but poor splits made me scrap the idea, however it was related to this [bug](#) in scikit-learn 1.7.2 that env was on, updated requirements.txt for scikit-learn>=1.8.0
 - While the pre-made splits do not have data leakage, when opting not to use them it uses the old splitting code that cannot handle grouped cases. Fixed by skipping over fake cases and adding them to the same set as their real case after splitting. The logic behind this is that it is not under the same constraints of make_splits.py and grouped cases are similar so they are more redundant and this is a fix with minimal changes [↗](#)

Quick Code Updates

- [run_splits_array.sbatch](#) is the Slurm job array friendly way to bulk submit jobs to train models with different combinations of splits and seeds, other code was made to work with it
- Slurm job arrays are preferred to submitting multiple jobs because it reduces load on scheduler and queue time
- An example job array task is below, corresponding to training one split fully
- Compared to the model at the beginning of the quarter, we are using 99.5% less cpu compute, 90% less memory, and finish in 97% less time
- Submitting 50 jobs requires 50 times the resource allocation, for --mem=128G that's 6.4T and queue times would be >30 minutes
- Minimize request to what you actually use for higher priority, recommendations: --partition=short, --ntasks=4, --mem=4G

```
[trd1531@quser43 ~]$ seff 2790466_0
Job ID: 2790494
Array Job ID: 2790466_0
Cluster: quest
User/Group: trd1531/trd1531
State: COMPLETED (exit code 0)
Nodes: 1
Cores per node: 8
CPU Utilized: 00:48:21
CPU Efficiency: 33.15% of 02:25:52 core-walltime
Job Wall-clock time: 00:18:14
Memory Utilized: 2.07 GB
Memory Efficiency: 25.87% of 8.00 GB (8.00 GB/node)
```

Mathlete

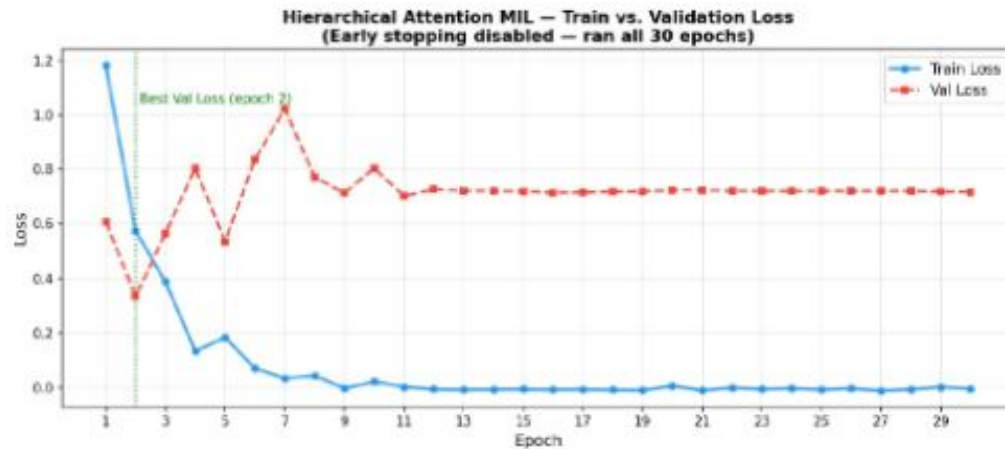
Abhi Vinnakota

Reflection

- Key learnings
 - Multiple Instance Learning
 - Knowledge on how to use Quest + shell commands
 - Using git with a team
- What I would do differently for a similar project
 - Have more knowledge of other roles for context, it will help my role
 - Make sure to communicate frequently with other team members
- Applications to real-world data science projects
 - Efficient use of git with a team in real-world
 - AI tools to bounce ML theory concepts off of

The Problem

Observed in every split



- Massively overparameterized model
 - $p \approx 2.4$ million parameters, trained on $n=63$ labeled cases, p/n ratio = 38,000
 - Underdetermined system with more parameters than constraints: solution always exists
- Labels are bag-level, not patch-level
 - Each of the cases gets one label, every patch inside that case shares it
 - More patches don't add a supervised signal, sample size is effectively n bags

MIL Setup

Notation (from MIL slides)

$X = \{x_1, \dots, x_K\}$	Bag of K patches
$y_i \in \{0, 1\}$	Instance label, unknown during training
$Y = \max\{y_i\}$	Bag label, the only supervision you get
$z = \sum_i a_i h_i$	Bag embedding: weighted avg of patch embeddings
$\hat{p} = \text{classifier}(z)$	Predicted probability for the bag
$\mathcal{L} = -[Y \log \hat{p} + (1-Y) \log(1-\hat{p})]$ Loss: one scalar per bag	

Matters because

- Hierarchical attention model uses gated attention
 - y_i isn't seen. Attention parameters must infer which patches matter from bag labels.
- Only 63 bags provide that signal

Why Model Fits Training Labels

- **A solution always exists**
 - With $p \gg n$, the constraint set $\{\mathcal{L}_i(\theta) = 0\}$ is underdetermined, more free parameters than equations
 - The model's output surface in $\mathbb{R}^n$ can pass through any n target values. An interpolating θ^* always exists.
- **SGD (batch size = 1) finds it**
 - Each update moves θ to reduce one bag's loss. With 2.4M parameters and 63 bags, nothing prevents the optimizer from reaching train loss ≈ 0 .
 - Different patch orderings $\rightarrow$ different gradient paths $\rightarrow$ different local minima, all with train loss = 0. This could explain the 79%–100% accuracy variance across splits
- **Solution doesn't generalize**
 - SGD lands in one θ^* based on patch ordering.
 - Most interpolating solutions are in sharp minima. Slight perturbation in input $\rightarrow$ large output change
 - A test case with different patch distributions is exactly such a perturbation. Val loss stays high.

Why More Patches Don't Add Label Information

- $z = \sum_i a_i h_i$ is one point in $\mathbb{R}^{512}$.
 - The model learns n such (z, Y) pairs - one per bag
- Each bag contributes one label, whether it has 500 or 5000 patches
- Gated attention amplifies this
 - 393,600 parameters alone for attention, outnumber training bags by $\sim 6,000\times$
 - Sigmoid gate adds a learned feature-masking axis on top of patch selection, giving the model more ways to separate training cases without learning anything generalizable

Connection to HierarchicalAttnMIL

- **patch projector** ($4096 \rightarrow 512$): $\sim 2.1\text{M}$ parameters
 - Primary parameter pool. With 2.1M params and 63 bags, can create arbitrarily different embeddings for each training case
- **Gated Attention** x 3: $\sim 393\text{K}$ parameters
 - 131K attention params per level vs. 63 bags $\rightarrow$ learns spurious routing easily. Sigmoid gate amplifies overfitting.
- **classifier** ($512 \rightarrow 2$): ~ 1000 parameters
 - Trivial to fit 63 labels once bag embeddings are sufficiently separated
- **Total $\sim 2.4\text{M}$** parameters
 - vs 63 training bags

Implications

- **Train loss $\rightarrow 0$ is inevitable**
 - $p \gg n$ means an interpolating solution always exists and SGD always finds one. Not fixable by tuning, follows model and dataset size
- **More patches per case can't fix generalization**
 - Patches improve z but don't add label constraints. The limit is the number of bags. More labeled cases would help; more patches per case won't.
- **Mitigations can constrain but cannot overcome $p \gg n$**
 - dropout (flatter minima), `weight_decay` (smaller weights), `entropy_reg`, `class_weights` (imbalance). All help at the margins.

Visualize the High Attention Patches

Zijin Zeng

Patching Code Explanation & Update

Objective 1: Verify Patch Placement Along the Tissue Boundary

- Extract epithelial contours from each stain image.
- Generate patches along the contour to cover the epithelial region.
- Visualize patch placement to verify:
 - coverage of epithelial tissue
 - amount of background included
 - consistency across stains

Objective 2: Highlight Model Attention on Important Patches

- Update adapted from STAT390_2026Winter_Team4_Presentation3
- Attention weights are obtained from the attention analysis output.
- The file weights_df (derived from top_effective_patches_per_case CSV) contains attention scores only for the most important patches.
- Patches with recorded weights are highlighted, while the remaining patches are assigned weight = 0.

Patching Code Explanation & Update

```
In [ ]: result, contours, final_squares, normals=process_contours((m[1]), epithelium_mask_2D, 0.2)
```

```
-----  
TypeError                                Traceback (most recent call last)  
Cell In[53], line 1  
----> 1 result, contours, final_squares, normals=process_contours((m[1]), epithelium_mask_2D, 0.2)  
  
TypeError: process_contours() missing 1 required positional argument: 'effective_weights'
```

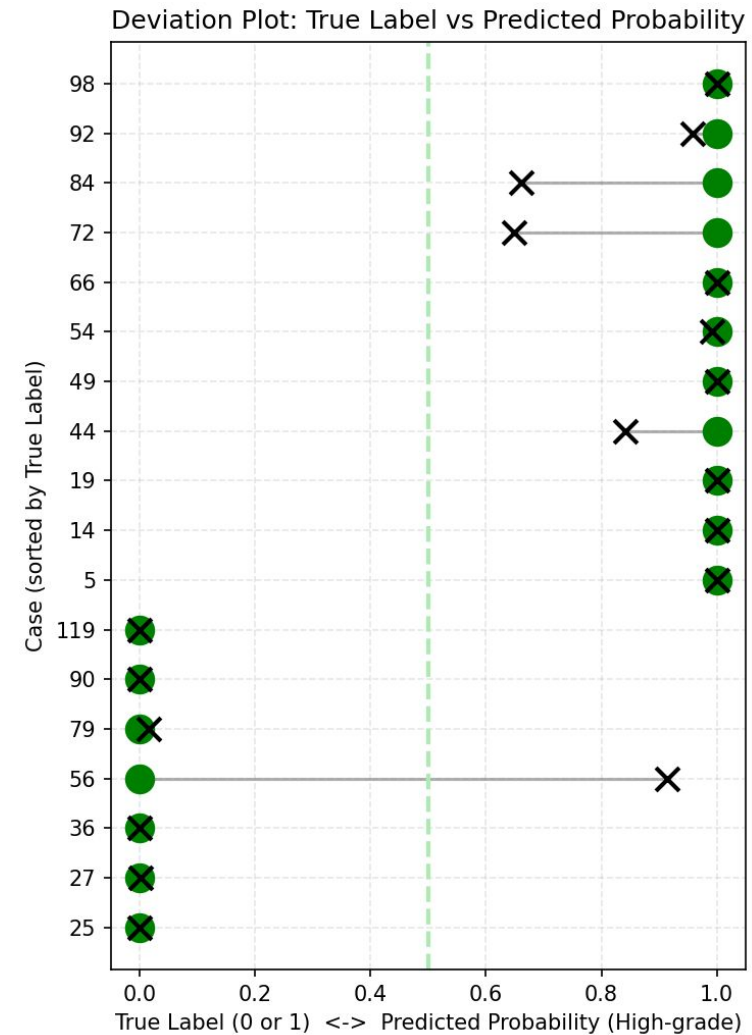
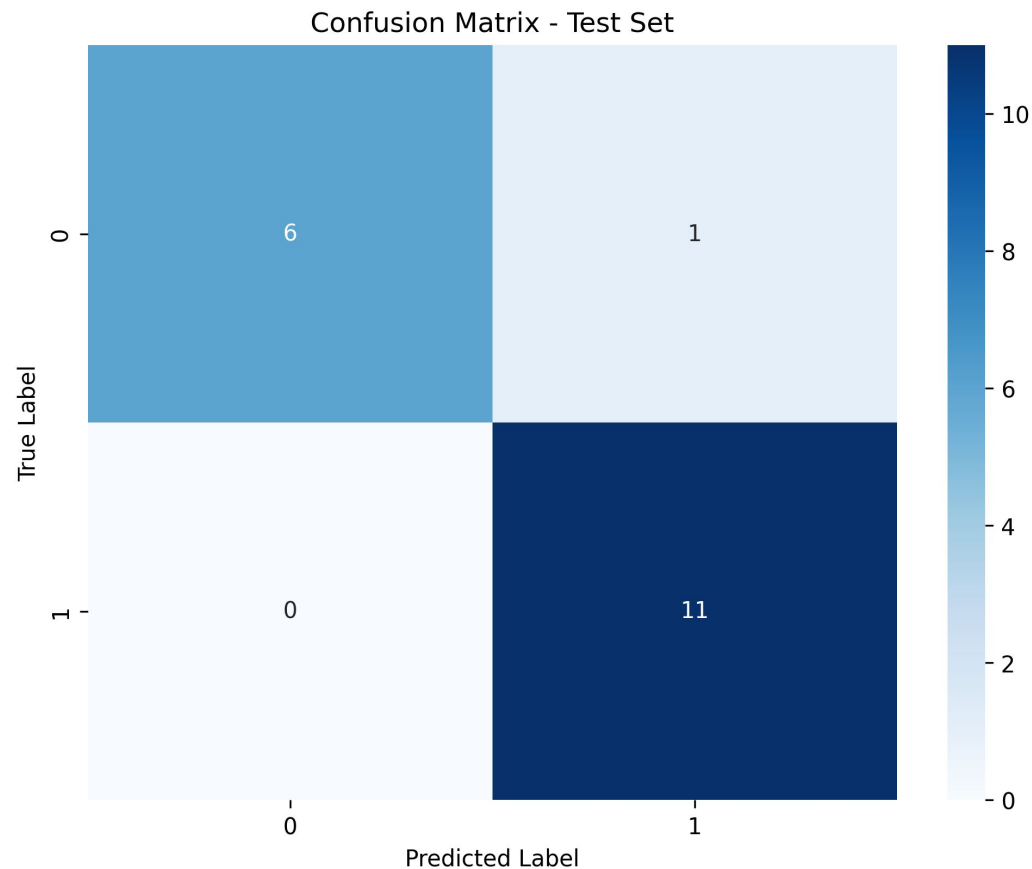
From STAT390_Winter2026/blob/master/Team4/Code/Pres3_Patching_Code_Attention_Bella.ipynb

Old function call: result, contours, final_squares, normals = process_contours(...)

New function call: result, contours, final_squares, normals, patch_length = process_contours(...)

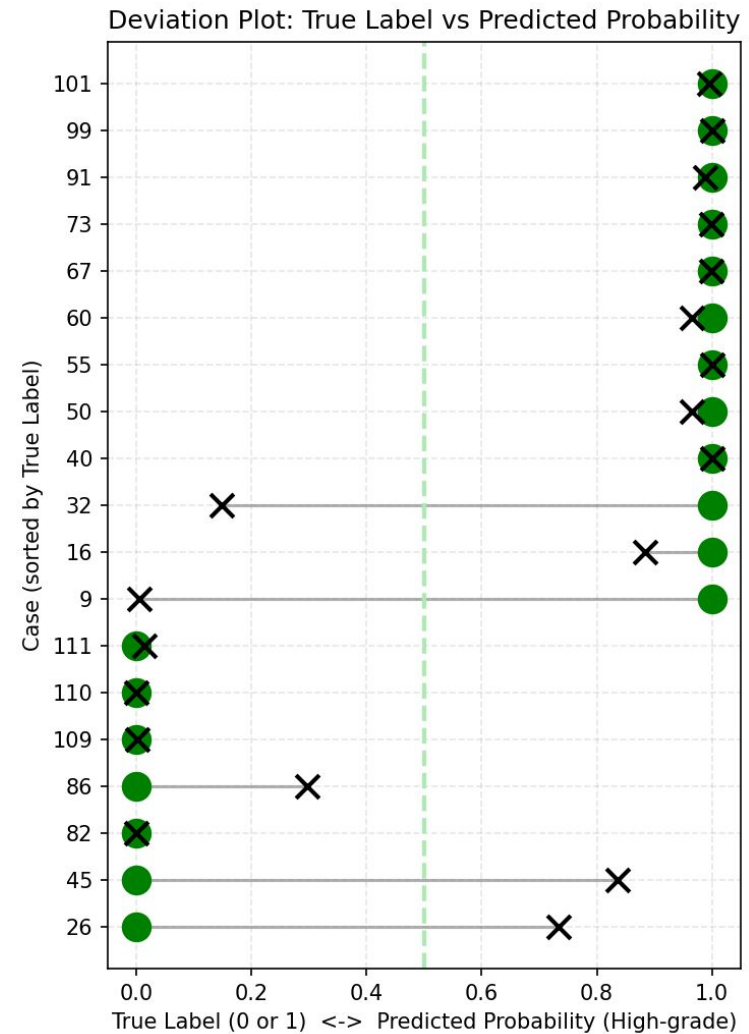
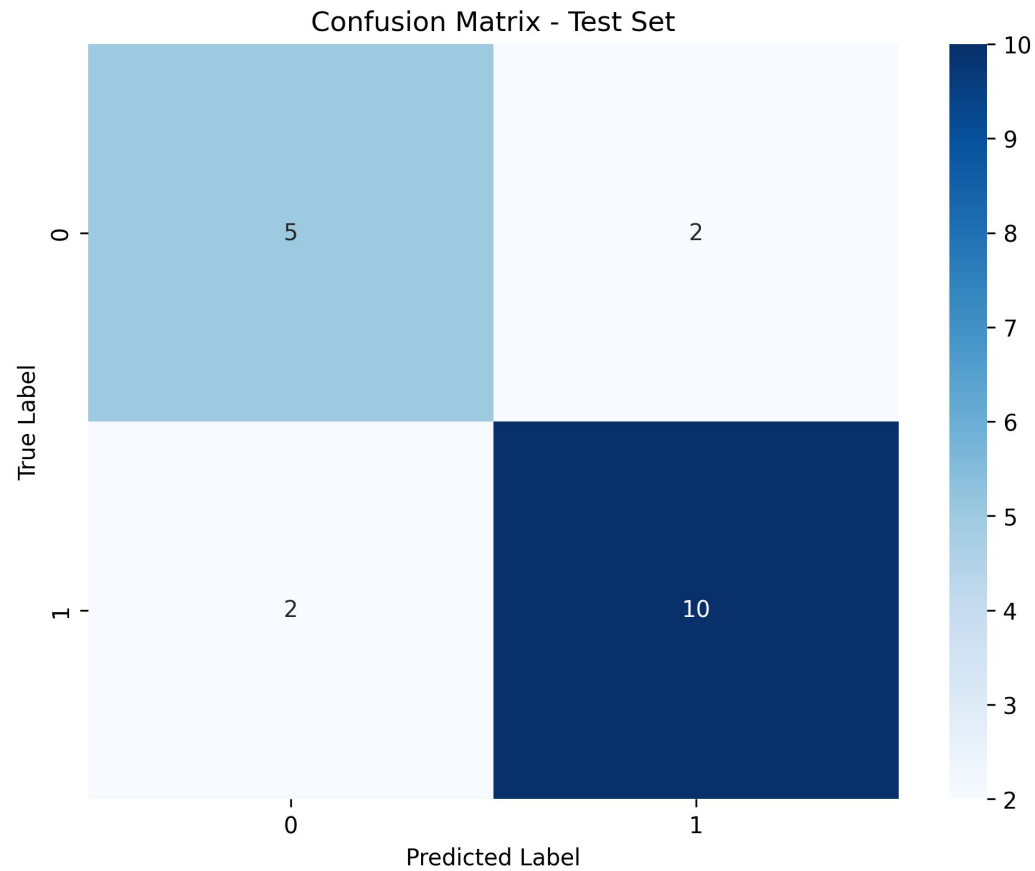
The updated process_contours() returns one additional output, patch_length_list, so the caller must unpack five values instead of four. The extra output stores the patch length information generated during patch creation.

Code9_no_leakage/sbatch_files/test.out



https://github.com/hchau3/STAT390_Winter2026_Team6/blob/main/Code4_patching_code_Zijin/visualization_Zijin.ipynb

Code9_no_leakage/sbatch_files/test2.out

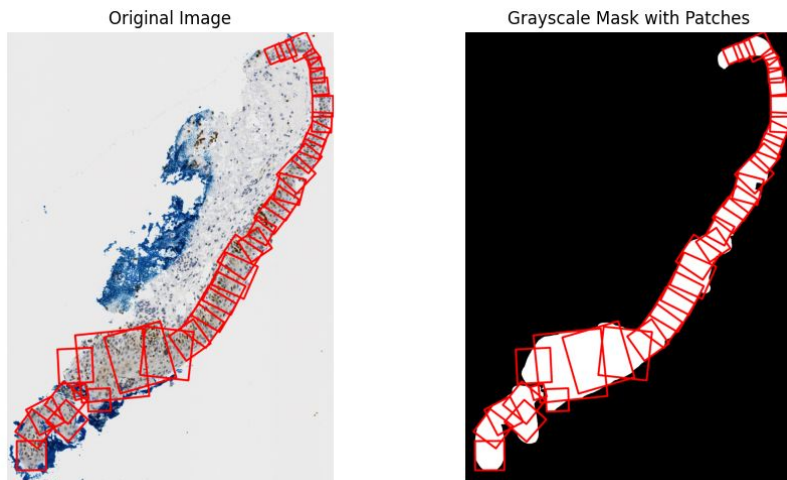


https://github.com/hchau3/STAT390_Winter2026_Team6/blob/main/Code4_patching_code_Zijin/visualization_Zijin.ipynb

Case 9

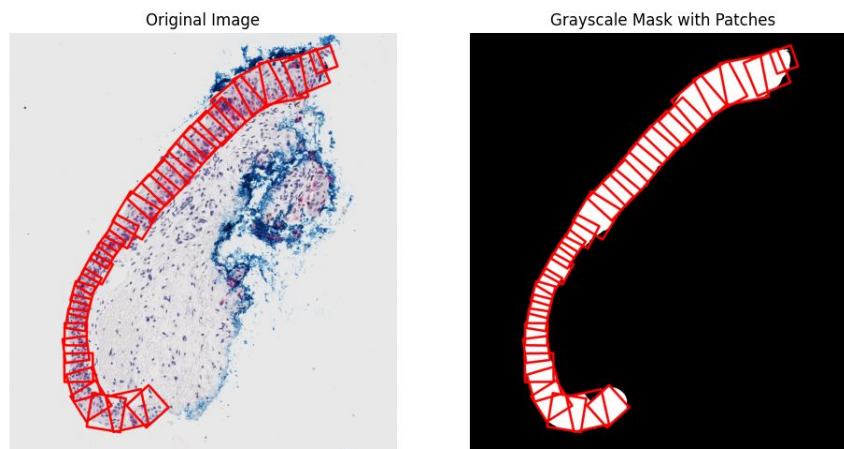
https://github.com/hchau3/STAT390_Winter2026_Team6/blob/main/Code4_patching_code_Zijin/test2/Pres4_Patching_Code_Attention_updated%202.ipynb

sox10



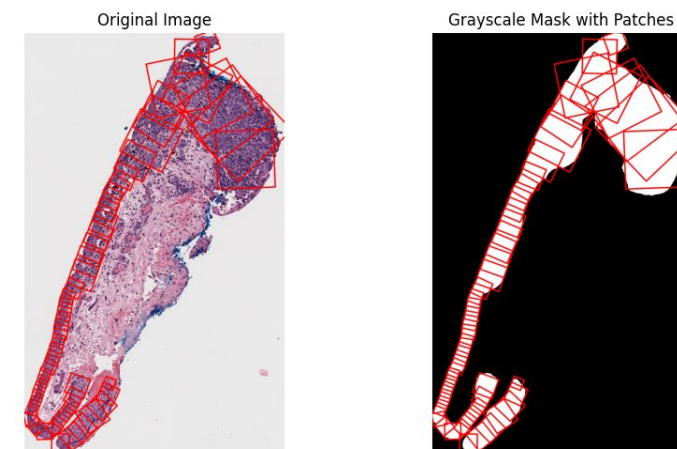
	Metric	Value
0	num_patches	35.000000
1	total_epithelium_coverage	97.059470
2	background_pixel_percent	9.585645
3	score	95.730447
4	extension_factor	1.100000
5	step_size	2.000000
6	max_width	2048.000000
7	overlap_threshold	0.300000
Sigma of : 20.0		

melan



	Metric	Value
0	num_patches	28.000000
1	total_epithelium_coverage	98.878952
2	background_pixel_percent	5.506386
3	score	98.001885
4	extension_factor	1.100000
5	step_size	2.000000
6	max_width	2048.000000
7	overlap_threshold	0.300000
Sigma of : 4.0		
Sigma of : 30.0		

h&e

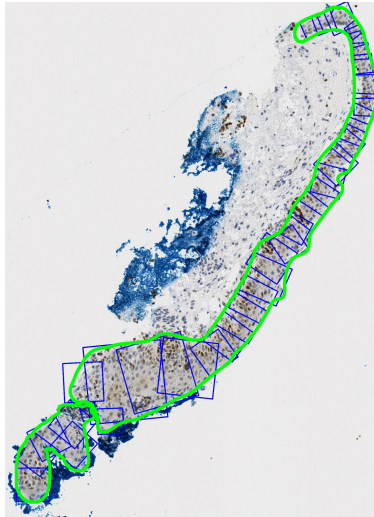


	Metric	Value
0	num_patches	53.000000
1	total_epithelium_coverage	97.969013
2	background_pixel_percent	9.644491
3	score	96.446312
4	extension_factor	1.100000
5	step_size	2.000000
6	max_width	2048.000000
7	overlap_threshold	0.300000

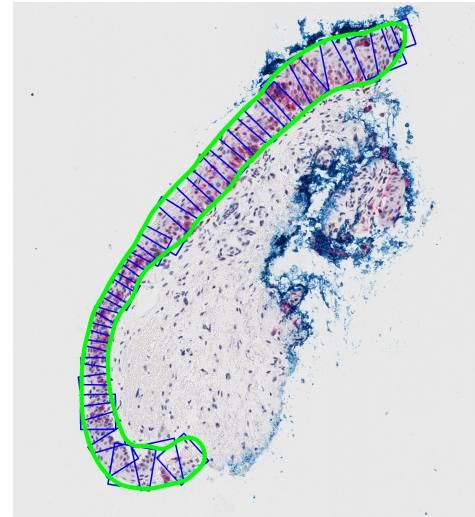
Case 9

https://github.com/hchau3/STAT390_Winter2026_Team6/blob/main/Code4_patching_code_Zijin/test2/Pres4_Patching_Code_Attention_updated%20.ipynb

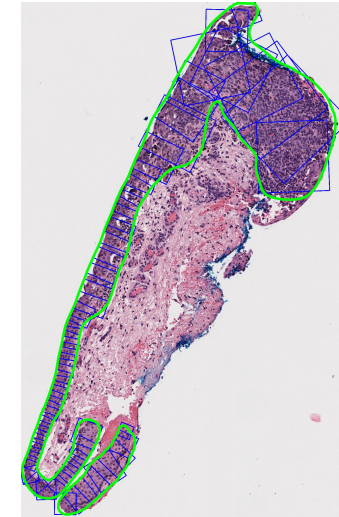
sox10



melan



h&e

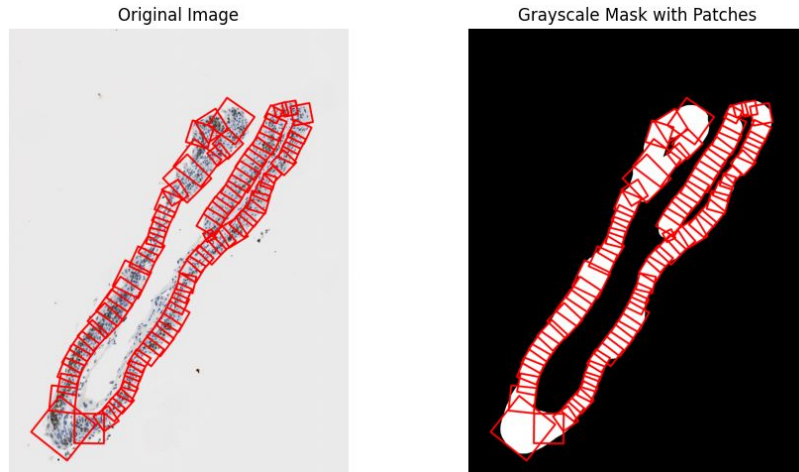


case_id	stain	slice_idx	patch_idx	patch_attn_weight	slice_attn_weight	stain_attn_weight	effective_weight
9	sox10	0	0	0.023052942	1	0.895274341	0.020638707
9	sox10	0	22	0.023369744	1	0.895274341	0.020922333
9	sox10	0	16	0.024314873	1	0.895274341	0.021768482
9	sox10	0	1	0.029692307	1	0.895274341	0.026582761
9	sox10	0	6	0.033557676	1	0.895274341	0.030043326
9	sox10	0	31	0.040998574	1	0.895274341	0.036704972
9	sox10	0	30	0.040998574	1	0.895274341	0.036704972
9	sox10	0	4	0.088161692	1	0.895274341	0.078928901
9	sox10	0	32	0.121351317	1	0.895274341	0.10864272
9	sox10	0	3	0.127242893	1	0.895274341	0.113917297
9	sox10	0	23	0.134787872	1	0.895274341	0.120672124

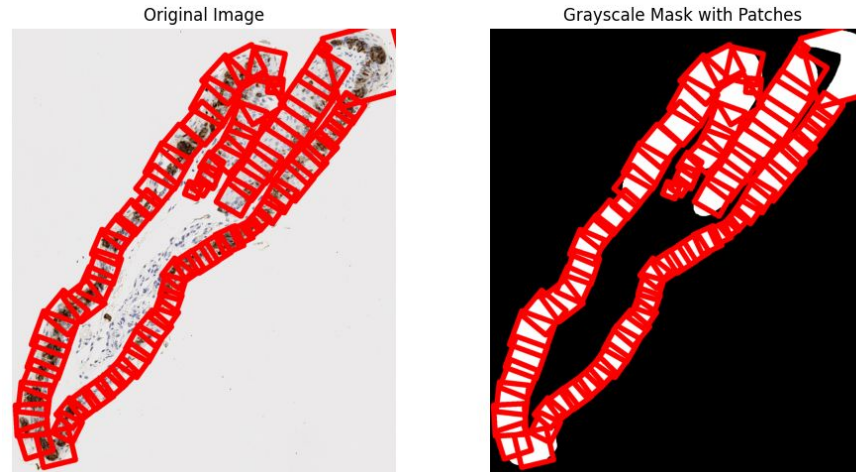
Case 32

https://github.com/hchau3/STAT390_Winter2026_Team6/blob/main/Code4_patching_code_Zijin/test2/Pres4_Patching_Code_Attention_updated%202.ipynb

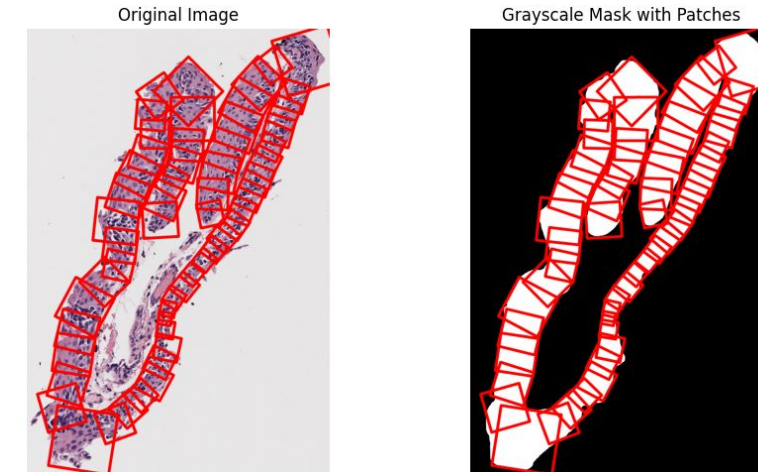
sox10



melan



h&e



	Metric	Value
0	num_patches	69.000000
1	total_epithelium_coverage	98.036774
2	background_pixel_percent	5.665701
3	score	97.296279
4	extension_factor	1.100000
5	step_size	2.000000
6	max_width	2048.000000
7	overlap_threshold	0.300000

Sigma of : 20.0

	Metric	Value
0	num_patches	69.000000
1	total_epithelium_coverage	98.154101
2	background_pixel_percent	8.780129
3	score	96.767255
4	extension_factor	1.100000
5	step_size	2.000000
6	max_width	2048.000000
7	overlap_threshold	0.300000

Sigma of : 26.0

	Metric	Value
0	num_patches	56.000000
1	total_epithelium_coverage	98.222111
2	background_pixel_percent	8.684953
3	score	96.840698
4	extension_factor	1.100000
5	step_size	2.000000
6	max_width	2048.000000
7	overlap_threshold	0.300000

Case 32

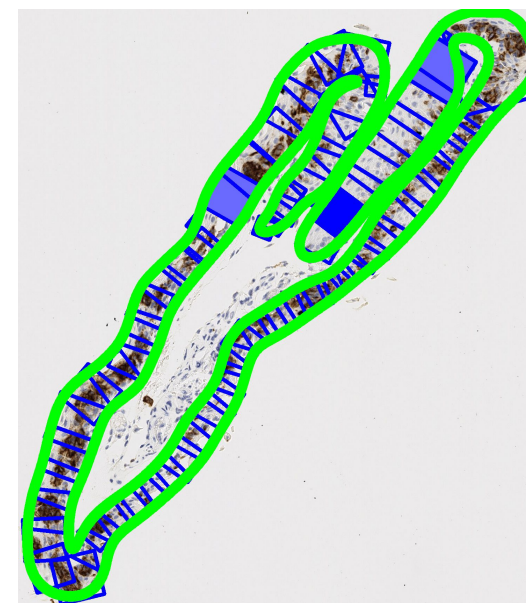
https://github.com/hchau3/STAT390_Winter2026_Team6/blob/main/Code4_patching_code_Zijin/test2/Pres4_Patching_Code_Attention_updated%202.ipynb

case_id	stain	slice_idx	patch_idx	patch_attn_weight	slice_attn_weight	stain_attn_weight	effective_weight
32	sox10	0	27	0.01561754	1	0.586569369	0.009160771
32	sox10	0	47	0.015680498	1	0.586569369	0.0091977
32	sox10	0	64	0.015720787	1	0.586569369	0.009221332
32	melan	1	53	0.042629045	0.539227724	0.407146543	0.009358981
32	sox10	0	58	0.016201455	1	0.586569369	0.009503277
32	melan	1	2	0.043709587	0.539227724	0.407146543	0.009596208
32	sox10	0	50	0.017078802	1	0.586569369	0.010017902
32	sox10	0	32	0.019445447	1	0.586569369	0.011406104
32	sox10	0	60	0.020674147	1	0.586569369	0.012126821
32	sox10	0	30	0.024511026	1	0.586569369	0.014377417
32	sox10	0	53	0.025902167	1	0.586569369	0.015193418
32	sox10	0	57	0.026729828	1	0.586569369	0.015678898
32	melan	1	7	0.073347308	0.539227724	0.407146543	0.016103013
32	sox10	0	13	0.02853634	1	0.586569369	0.016738543
32	melan	0	52	0.090822056	0.460772246	0.407146543	0.017038384
32	sox10	0	12	0.030938856	1	0.586569369	0.018147785
32	sox10	0	56	0.034064274	1	0.586569369	0.01998106
32	melan	0	22	0.195495591	0.460772246	0.407146543	0.03667533
32	sox10	0	34	0.150021523	1	0.586569369	0.08799803

sox10



melan



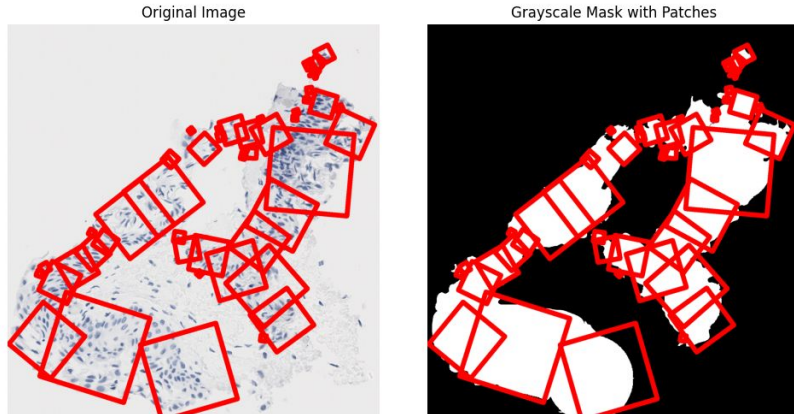
h&e



Case 45

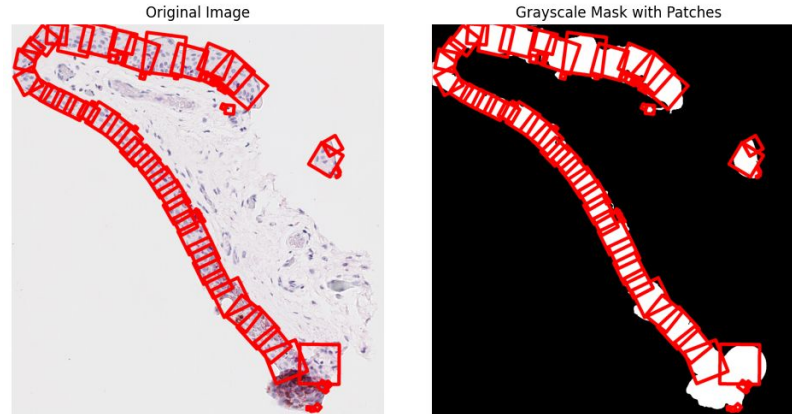
https://github.com/hchau3/STAT390_Winter2026_Team6/blob/main/Code4_patching_code_Zijin/test2/Pres4_Patching_Code_Attention_updated%202.ipynb

sox10



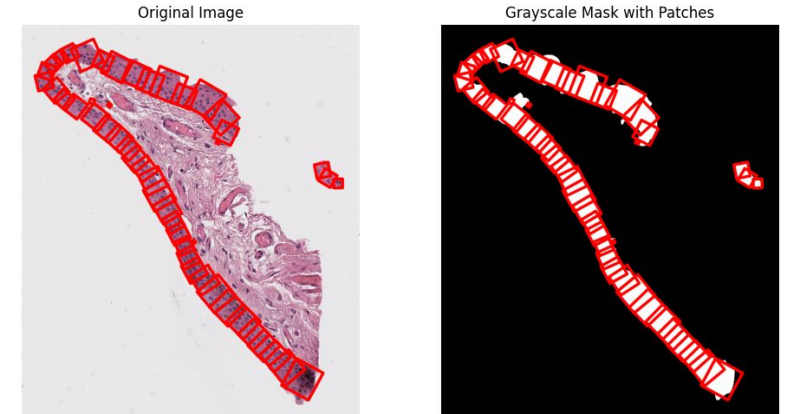
	Metric	Value
0	num_patches	54.000000
1	total_epithelium_coverage	85.764029
2	background_pixel_percent	12.373837
3	score	86.136456
4	extension_factor	1.100000
5	step_size	2.000000
6	max_width	2048.000000
7	overlap_threshold	0.300000

melan



	Metric	Value
0	num_patches	74.000000
1	total_epithelium_coverage	88.071514
2	background_pixel_percent	8.620651
3	score	88.733081
4	extension_factor	1.100000
5	step_size	2.000000
6	max_width	2048.000000
7	overlap_threshold	0.300000
Sigma of : 2.0		
Sigma of : 33.0		

h&e

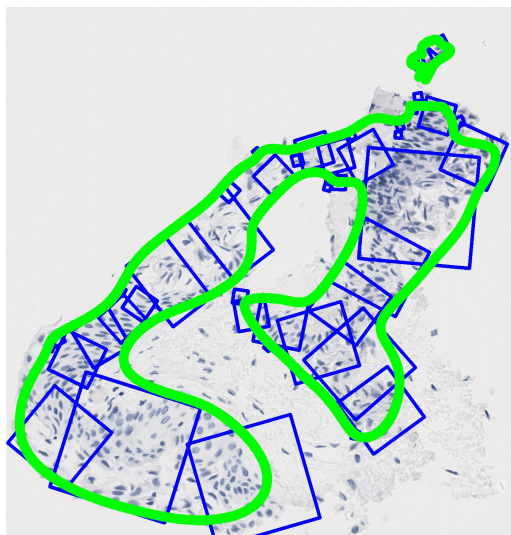


	Metric	Value
0	num_patches	48.000000
1	total_epithelium_coverage	94.595068
2	background_pixel_percent	7.492283
3	score	94.177597
4	extension_factor	1.100000
5	step_size	2.000000
6	max_width	2048.000000
7	overlap_threshold	0.300000
Sigma of : 38.0		
Sigma of : 2.0		

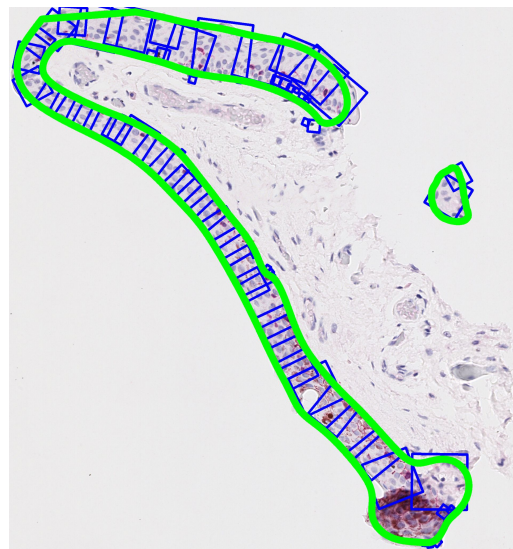
Case 45

https://github.com/hchau3/STAT390_Winter2026_Team6/blob/main/Code4_patching_code_Zijin/test2/Pres4_Patching_Code_Attention_updated%202.ipynb

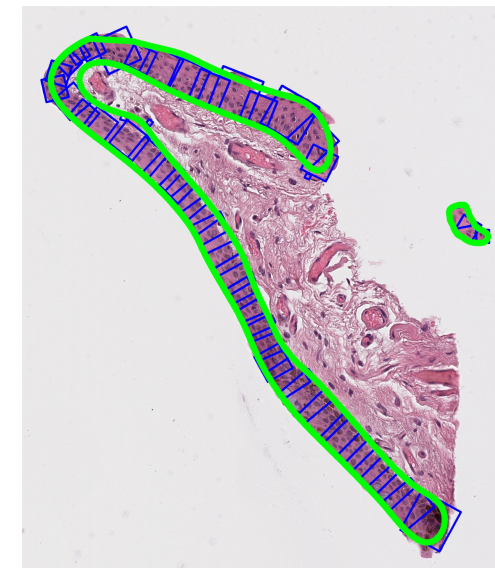
sox10



melan



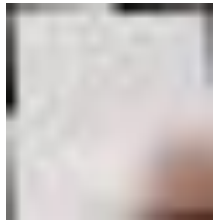
h&e



case_id	stain	slice_idx	patch_idx	patch_attn_weight	slice_attn_weight	stain_attn_weight	effective_weight
45	melan	0	22	0.031899609	1	0.992190301	0.031650482
45	melan	0	15	0.066637658	1	0.992190301	0.066117238
45	melan	0	27	0.1162901	1	0.992190301	0.115381909
45	melan	0	24	0.11763946	1	0.992190301	0.116720731
45	melan	0	29	0.127709895	1	0.992190301	0.12671252
45	melan	0	26	0.158609822	1	0.992190301	0.157371127

Case 45

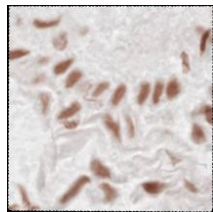
sox10



Color Model **RGB**
Depth **8**
Pixel Height **21**
Pixel Width **20**



Color Model **RGB**
Depth **8**
Pixel Height **23**
Pixel Width **23**



Color Model **RGB**
Depth **8**
Pixel Height **212**
Pixel Width **212**

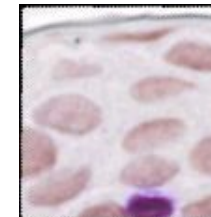
melan



Color Model **RGB**
Depth **8**
Pixel Height **10**
Pixel Width **10**

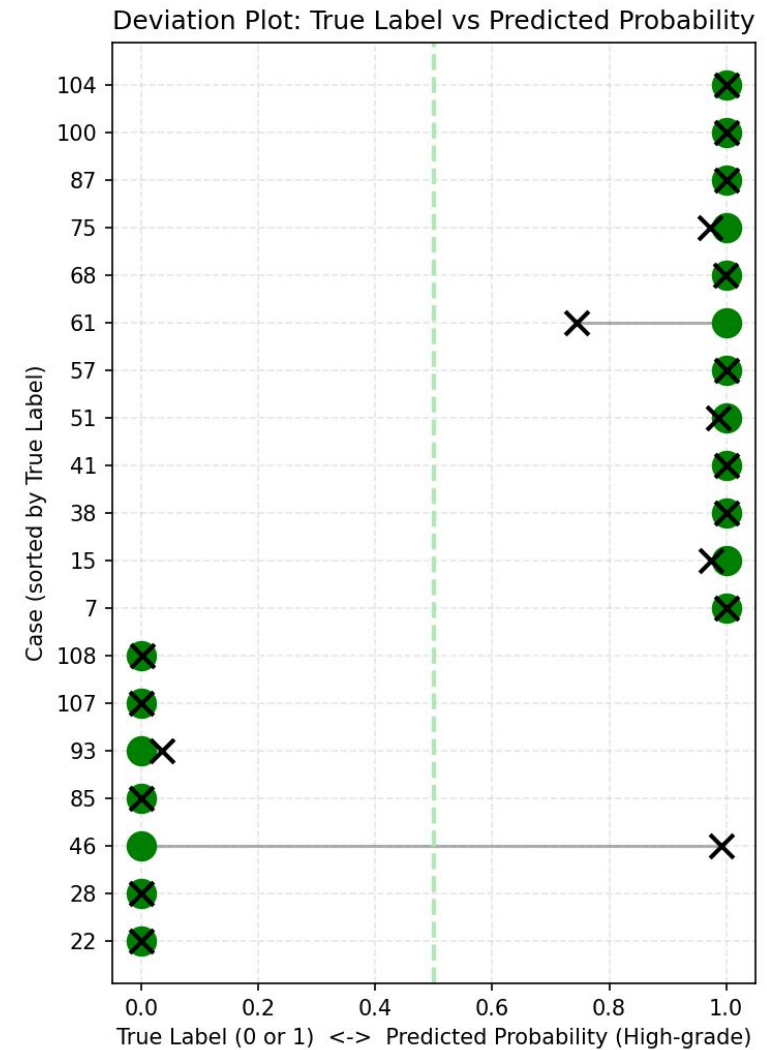
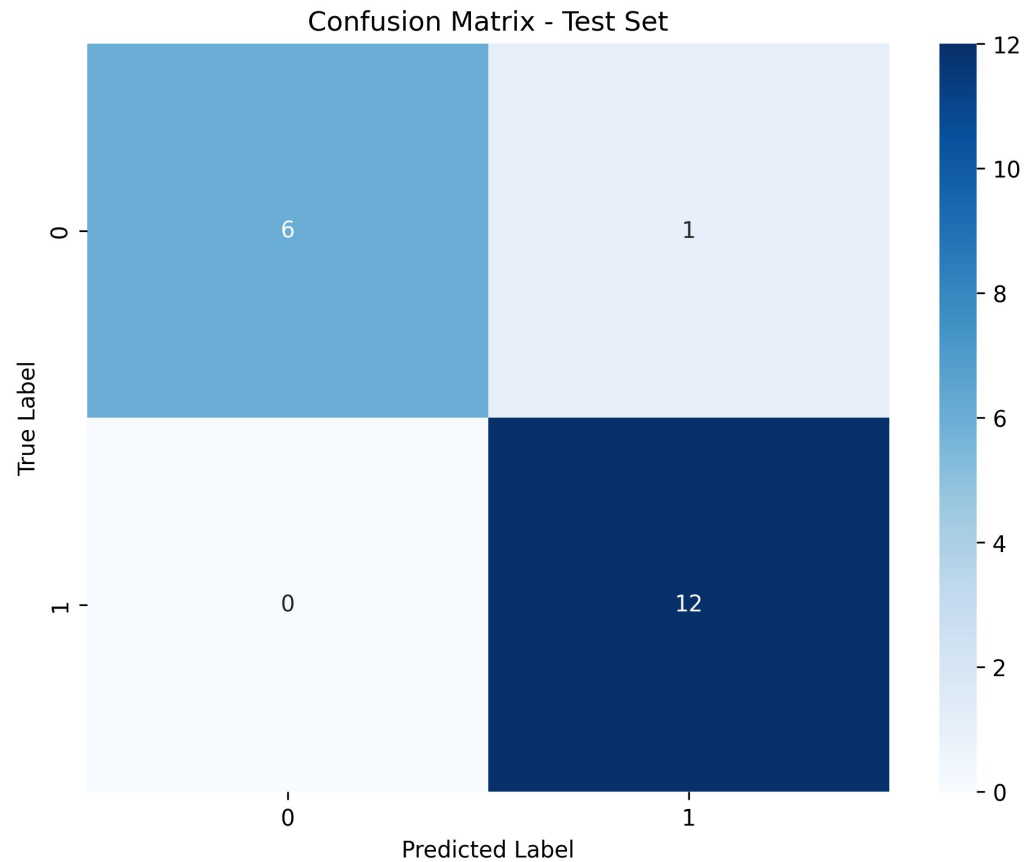


Color Model **RGB**
Depth **8**
Pixel Height **4**
Pixel Width **5**



Color Model **RGB**
Depth **8**
Pixel Height **116**
Pixel Width **116**

Code9_no_leakage/sbatch_files/test3.out

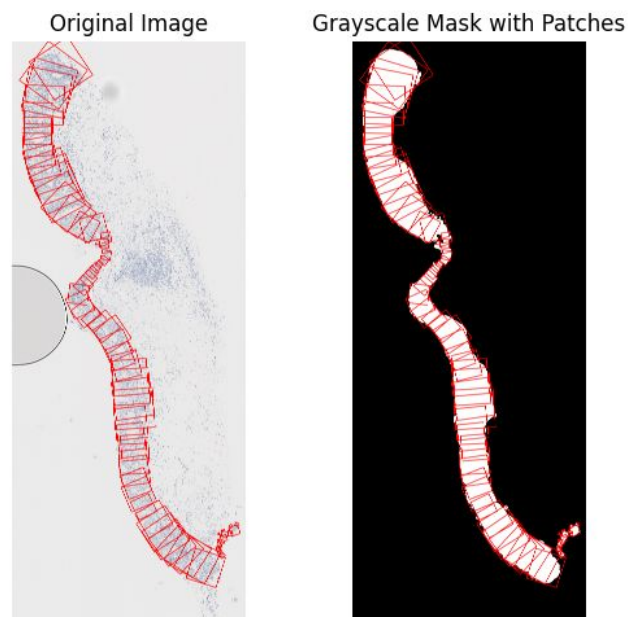


https://github.com/hchau3/STAT390_Winter2026_Team6/blob/main/Code4_patching_code_Zijin/visualization_Zijin.ipynb

Case 46

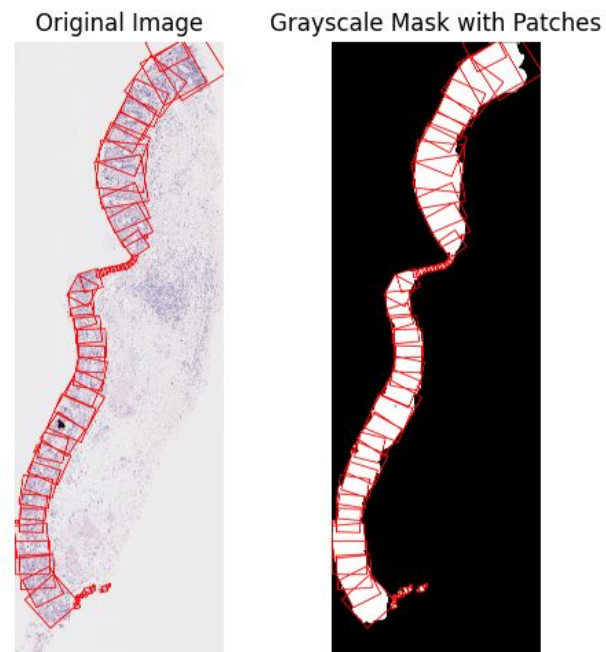
https://github.com/hchau3/STAT390_Winter2026_Team6/blob/main/Code4_patching_code_Zijin/test3/Pres4_Patching_Code_Attention_updated.ipynb

sox10



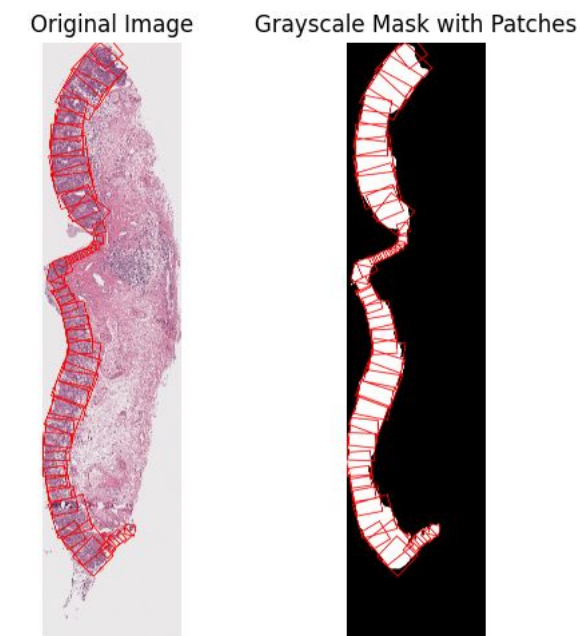
	Metric	Value
0	num_patches	91.000000
1	total_epithelium_coverage	99.085619
2	background_pixel_percent	12.416931
3	score	96.785109
4	extension_factor	1.100000
5	step_size	2.000000
6	max_width	2048.000000
7	overlap_threshold	0.300000
Sigma of :		58.0

melan



	Metric	Value
0	num_patches	67.000000
1	total_epithelium_coverage	99.034427
2	background_pixel_percent	11.836140
3	score	96.860314
4	extension_factor	1.100000
5	step_size	2.000000
6	max_width	2048.000000
7	overlap_threshold	0.300000
Sigma of :		6.0
Sigma of :		4.0
Sigma of :		63.0

h&e



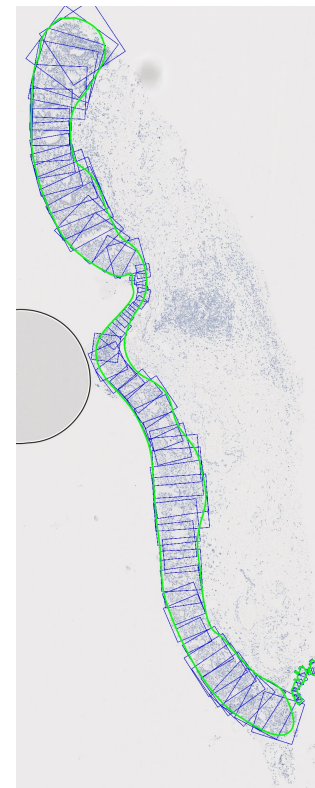
	Metric	Value
0	num_patches	51.000000
1	total_epithelium_coverage	98.772075
2	background_pixel_percent	8.915327
3	score	97.234595
4	extension_factor	1.100000
5	step_size	2.000000
6	max_width	2048.000000
7	overlap_threshold	0.300000

Case 46

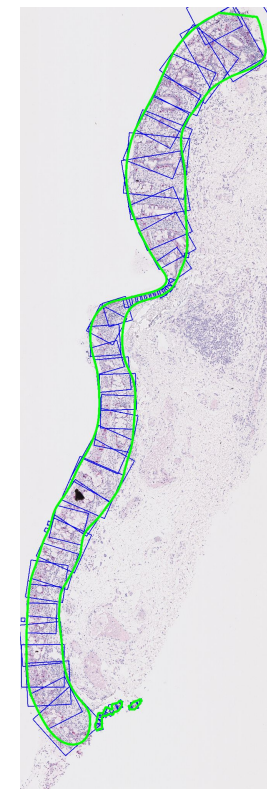
https://github.com/hchau3/STAT390_Winter2026_Team6/blob/main/Code4_patching_code_Zijin/test3/Pres4_Patching_Code_Attention_updated.ipynb

case_id	stain	slice_idx	patch_idx	patch_attn_weight	slice_attn_weight	stain_attn_weight	effective_weight
46	h&e	0	46	0.02436438	1	0.584315717	0.01423649
46	h&e	0	27	0.026410047	1	0.584315717	0.015431806
46	h&e	0	21	0.028444881	1	0.584315717	0.016620791
46	h&e	0	39	0.031376734	1	0.584315717	0.018333919
46	h&e	0	41	0.046777248	1	0.584315717	0.027332681
46	h&e	0	43	0.101291299	1	0.584315717	0.059186098
46	h&e	0	40	0.109734215	1	0.584315717	0.064119426
46	melan	0	7	0.246867791	1	0.318121105	0.078533855
46	h&e	0	42	0.15697284	1	0.584315717	0.091721698

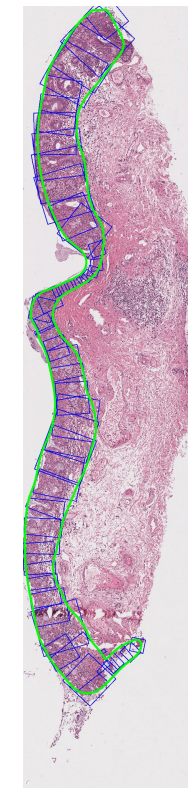
sox10



melan

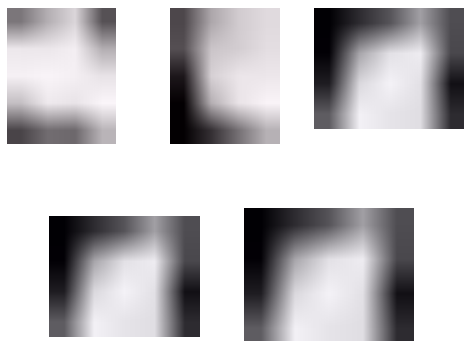


h&e



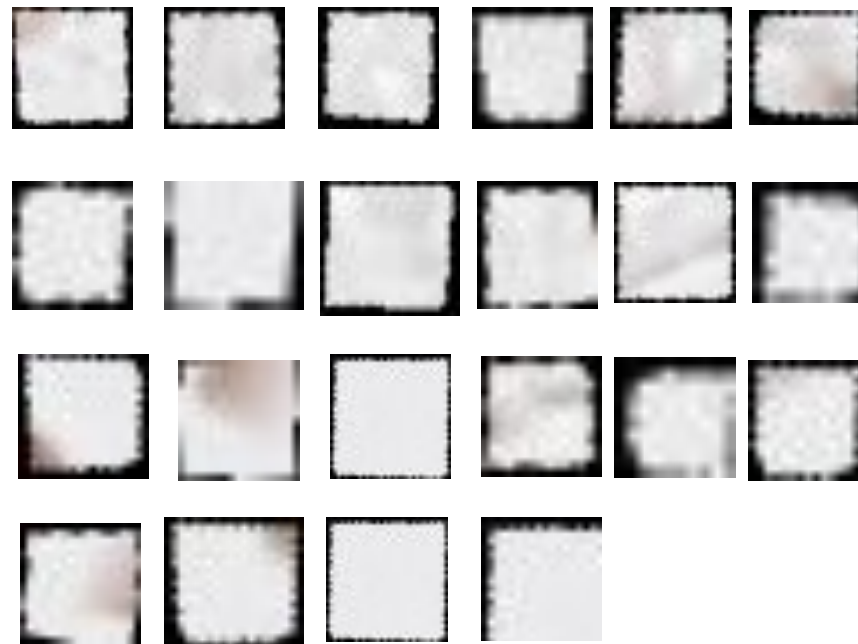
Case 46

melan



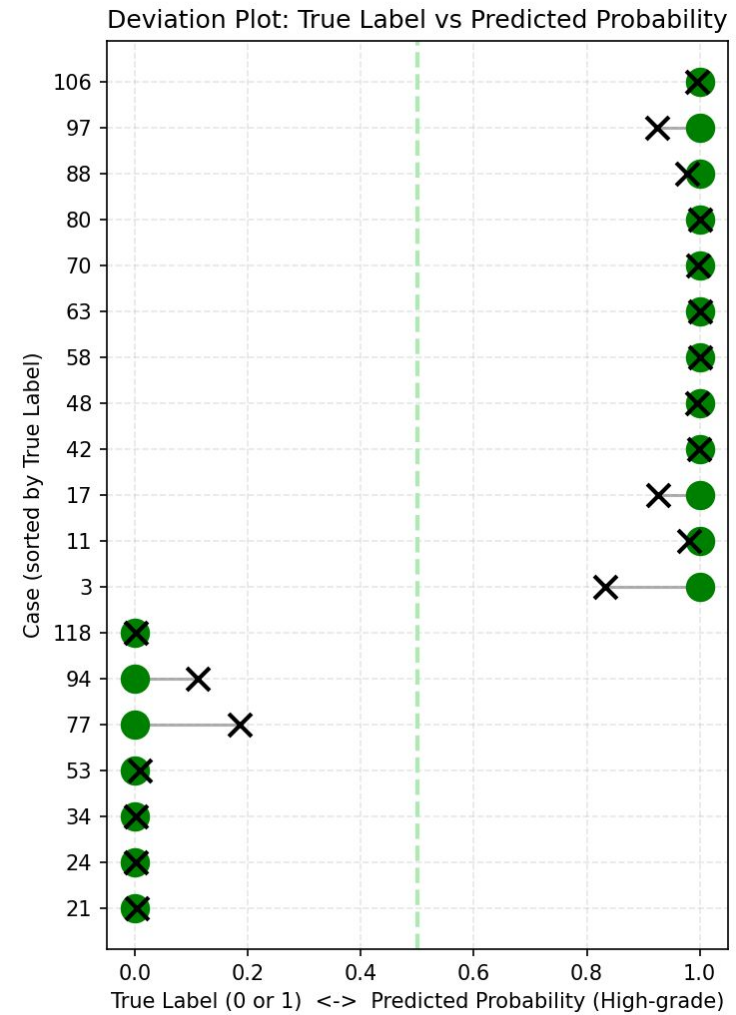
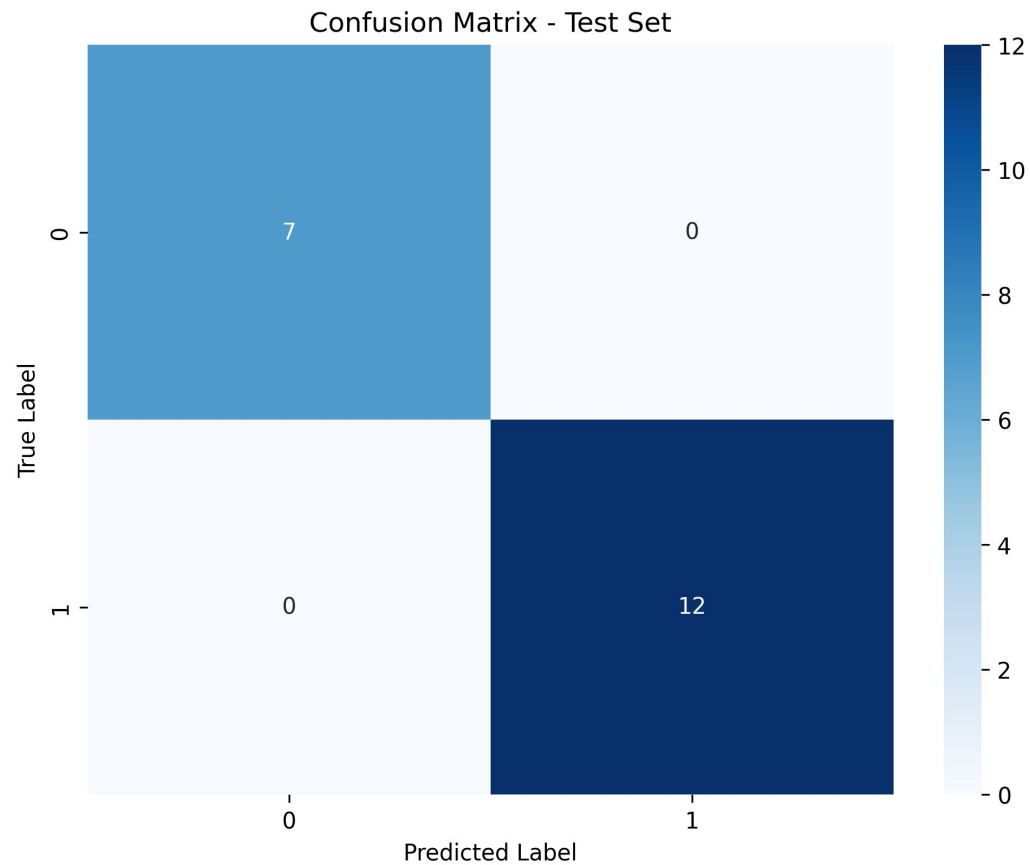
patch 11, 12, 15,
16, 17

sox10



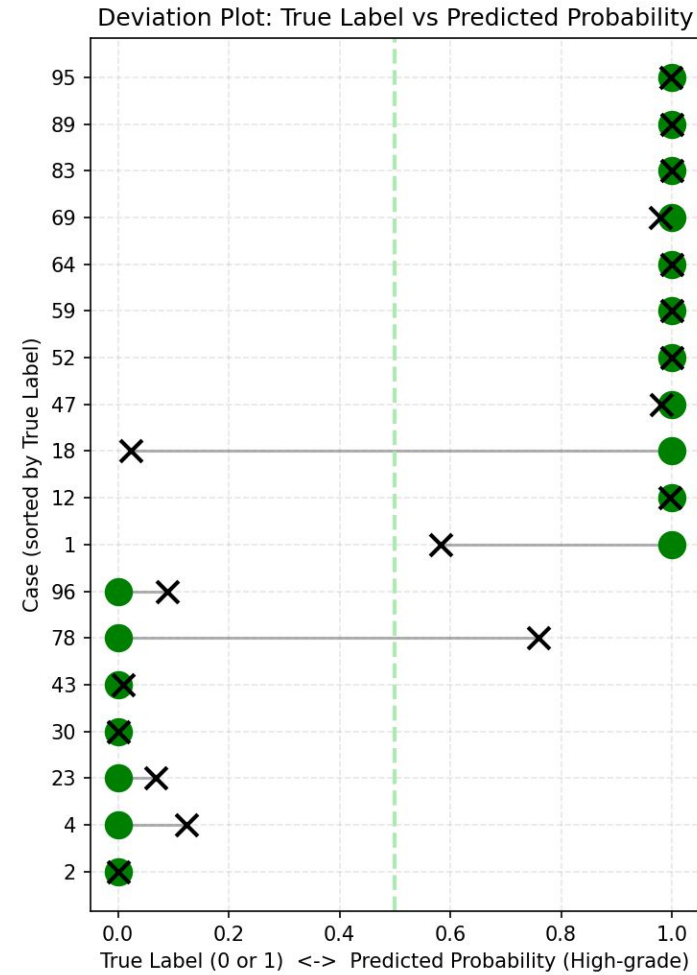
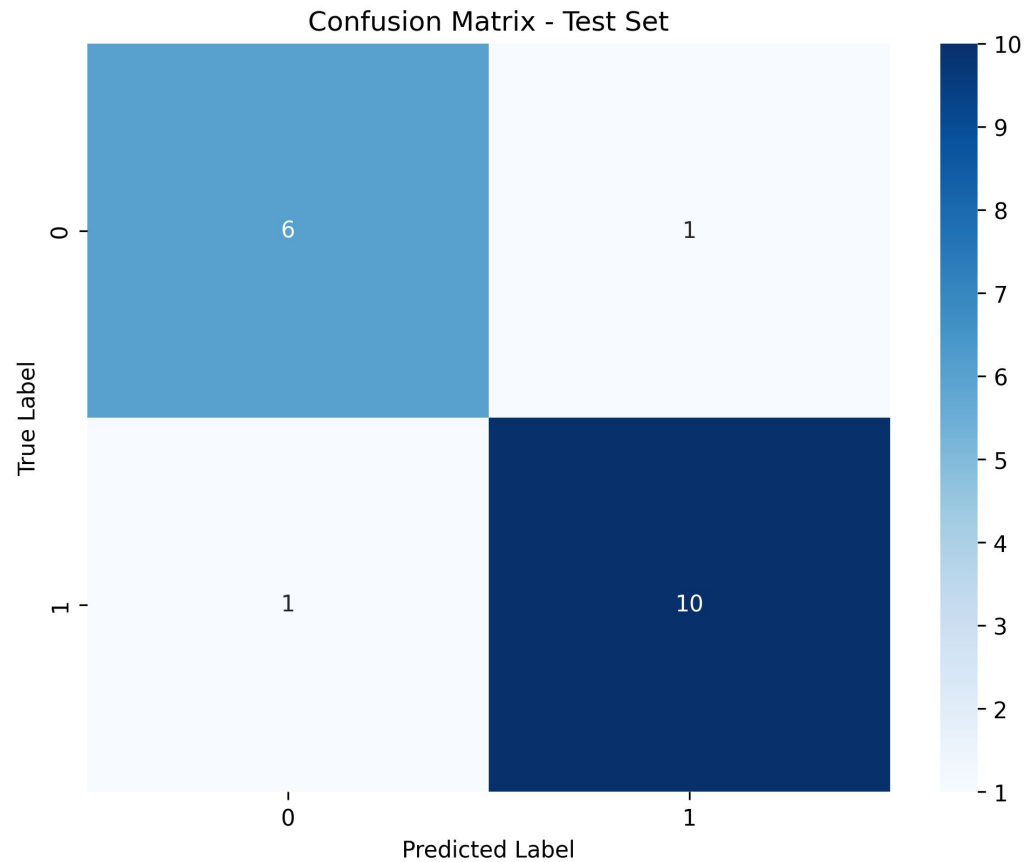
patch 0, 1, 2, 3, 4, 5, 8, 9, 10, 11, 14, 15,
16, 17, 18, 19, 20, 21, 22, 32, 33, 34

Code9_no_leakage/sbatch_files/test4.out



https://github.com/hchau3/STAT390_Winter2026_Team6/blob/main/Code4_patching_code_Zijin/visualization_Zijin.ipynb

Code9_no_leakage/sbatch_files/test5.out



https://github.com/hchau3/STAT390_Winter2026_Team6/blob/main/Code4_patching_code_Zijin/visualization_Zijin.ipynb

Case 78

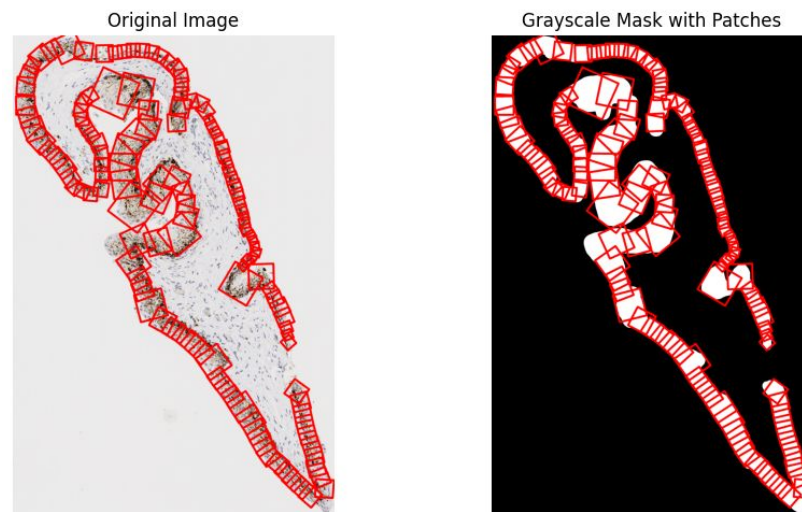
https://github.com/hchau3/STAT390_Winter2026_Team6/blob/main/Code4_patching_code_Zijin/test5/Pres4_Patching_Code_Attention_updated.ipynb

sox10



	Metric	Value
0	num_patches	103.000000
1	total_epithelium_coverage	96.594970
2	background_pixel_percent	8.653194
3	score	95.545337
4	extension_factor	1.100000
5	step_size	2.000000
6	max_width	2048.000000
7	overlap_threshold	0.300000

melan



	Metric	Value
0	num_patches	129.000000
1	total_epithelium_coverage	94.450739
2	background_pixel_percent	7.610925
3	score	94.038406
4	extension_factor	1.100000
5	step_size	2.000000
6	max_width	2048.000000
7	overlap_threshold	0.300000
Sigma of : 20.0		
Sigma of : 12.0		

h&e



	Metric	Value
0	num_patches	151.000000
1	total_epithelium_coverage	92.019311
2	background_pixel_percent	6.595073
3	score	92.296434
4	extension_factor	1.100000
5	step_size	2.000000
6	max_width	2048.000000
7	overlap_threshold	0.300000
Sigma of : 38.0		

Case 78

https://github.com/hchau3/STAT390_Winter2026_Team6/blob/main/Code4_patching_code_Zijin/test5/Pres4_Patching_Code_Attention_updated.ipynb

case_id	stain	slice_idx	patch_idx	patch_attn_weight	slice_attn_weight	stain_attn_weight	effective_weight
78	melan	0	0	0.023833938	1	0.236718312	0.00564193
78	melan	0	80	0.025126159	1	0.236718312	0.005947822
78	melan	0	68	0.026272634	1	0.236718312	0.006219214
78	melan	0	78	0.028887307	1	0.236718312	0.006838155
78	melan	0	76	0.029670147	1	0.236718312	0.007023467
78	sox10	0	20	0.009800827	1	0.76180476	0.007466317
78	sox10	0	10	0.009823833	1	0.76180476	0.007483842
78	melan	0	71	0.03511342	1	0.236718312	0.00831199
78	sox10	0	22	0.010998396	1	0.76180476	0.008378631
78	sox10	0	69	0.012548539	1	0.76180476	0.009559537
78	melan	0	15	0.041403323	1	0.236718312	0.009800925
78	melan	0	34	0.04284111	1	0.236718312	0.010141275
78	sox10	0	68	0.015988134	1	0.76180476	0.012179836
78	sox10	0	14	0.024837788	1	0.76180476	0.018921545
78	sox10	0	56	0.024974732	1	0.76180476	0.019025869
78	melan	0	95	0.095900074	1	0.236718312	0.022701304
78	melan	0	49	0.097951509	1	0.236718312	0.023186916
78	sox10	0	57	0.120767266	1	0.76180476	0.092001078
78	sox10	0	73	0.132165104	1	0.76180476	0.100684005

sox10



melan



h&e



Possible Problems / My Concerns

From STAT390_CMIL_Fall2025/blob/main/Code/Patching%20code/Patching_Code_Attention_Annika.ipynb

Parse information from filename, which provides: case_id, stain, and match index

```
filename = os.path.basename(slice_name)
match = re.match(r"case_(\d+)_match_(\d+)_(\h&e|melan|sox10)", filename)

case_number = int(match.group(1))
match_idx = int(match.group(2))
stain = match.group(3)
```

Select weights using only case and stain: retrieves all weights belonging to the same case and stain, regardless of which slice they belong to.

```
subset = (
    weights_df[
        (weights_df["case_id"] == case_number) &
        (weights_df["stain"] == stain)
    ]
    .sort_values("patch_idx", ascending=True)
)
```

Convert weights into an array, and then map weights to patches by order.

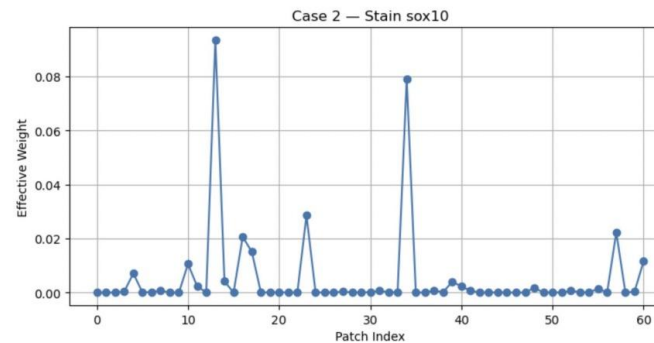
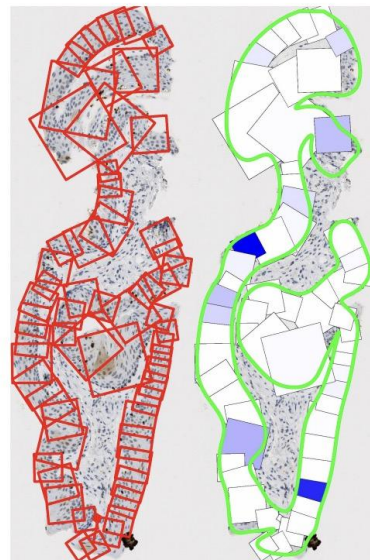
Possible Problems / My Concerns

From Presentation 5 - Fall 2025

Did not explicitly match slice_idx. This may work if only one slice exists. But becomes unreliable when multiple slices exist for the same stain

Benign cases: predicted correctly

Case 2 Sox 10



Possible Problems / My Concerns

From STAT390_Winter2026/blob/master/Team4/Code/Pres3_Patching_Code_Attention_Bella.ipynb

The new code does match at the slice level:

```
filename = os.path.basename(slice_name)
match = re.match(r"case_(\d+)_match_(\d+)_(\h&e|melan|sox10)", filename)

case_number = int(match.group(1))
match_idx = int(match.group(2))
stain = match.group(3)

subset = (
    weights_df[
        (weights_df["case_id"] == case_number) &
        (weights_df["stain"] == stain)
    ]
    .sort_values("patch_idx", ascending=True)
)
```

```
# Process contours and generate patch squares
overlay_weighted, contours, final_squares, normals, patch_length = process_contours(
    slice_details,
    binary,
    overlap_thresh,
    case_id=case_number,      # <-- FIXED
    stain=stain,
    slice_idx=match_idx,     # <-- FIXED
    weights_df=weights_df
)
```

However, it sets: `slice_idx = match_idx`. This assumes filename-based labels and attention-analysis slice indices use the same numbering system.

eg. filename `match_1` = weights_df slice_idx 1 → **incorrect**

Possible Problems / My Concerns

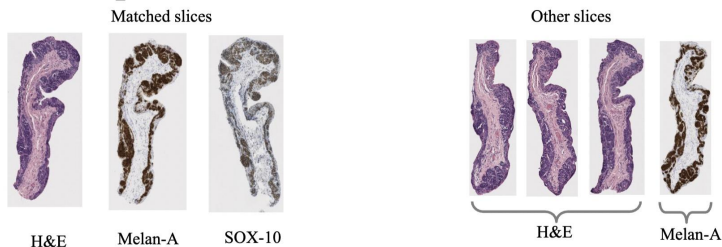
From attention analysis.py

In the attention-analysis pipeline, `slice\_idx` is created internally using `enumerate(...)` over the list of slices for each stain.

Therefore:

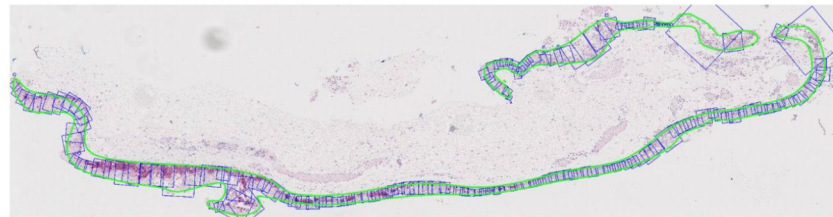
- first slice → slice_idx = 0
- second slice → slice_idx = 1
- third slice → slice_idx = 2

Identify slices that have similar shape across stains for patient h2114153



Attention Considered, Final Output: Case 84

Isabella



- Little/no fill because melan patches do not have high attention weights (as seen in table)
- All patches from Case 84, melan stain
 - Taken from "attention_analysis" CSV output of each run
 - top_effective_patches_per_case_5.0pct.csv
 - **RED** = highest attention weight across all other patches and stains for Case 84

Github:
STAT390_Winter2026/Team4/Code
/Pres3_Patching_Code_Attention_Bella.ipynb

Case	Stain	Weighted Attention
84	melan	0.002981174
84	melan	0.003220398
84	melan	0.003349242
84	melan	0.003652399
84	melan	0.003660225
84	melan	0.003719665
84	melan	0.006545869
84	melan	0.006658866
84	melan	0.007937271
84	melan	0.008595999
84	melan	0.009217261
84	melan	0.018107189
84	melan	0.019455414
84	melan	0.025855875
84	melan	0.032253165
84	melan	0.03453923
84	melan	0.065737151

From Team 4 Presentation 3 - 2026 Winter